Laboratorio di Elettronica Digitale

- PAL: Programmable Array Logic -

Programmazione a livello basso => si programmano le porte logiche

vanno su applicazioni che costano (civile o militare che sia)

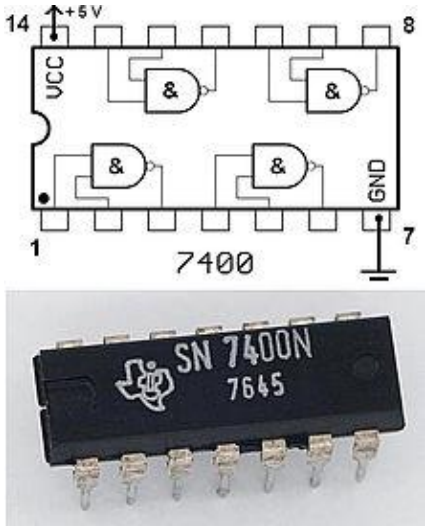
UNIVERSITÀ DEGLI STUDI FIRENZE
DINFO
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

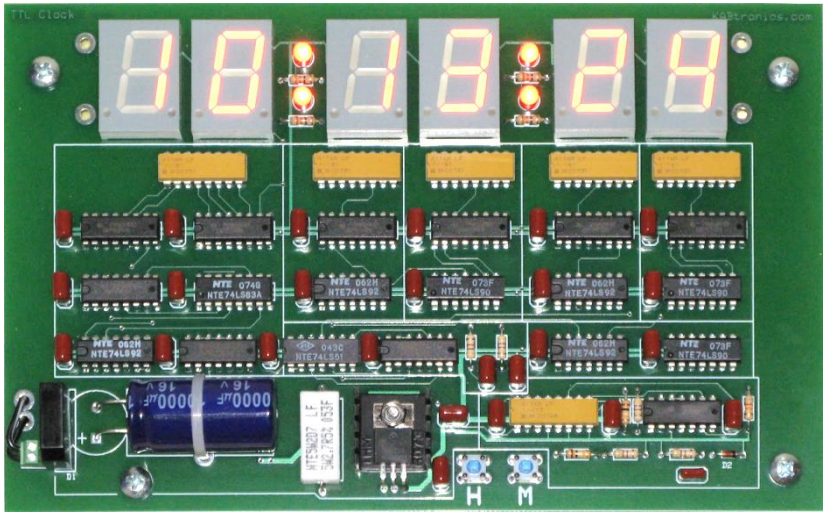
Updated 03/2025

- How logic circuits can be used ?

  Circuits exist(ed) that integrates simple logic functions

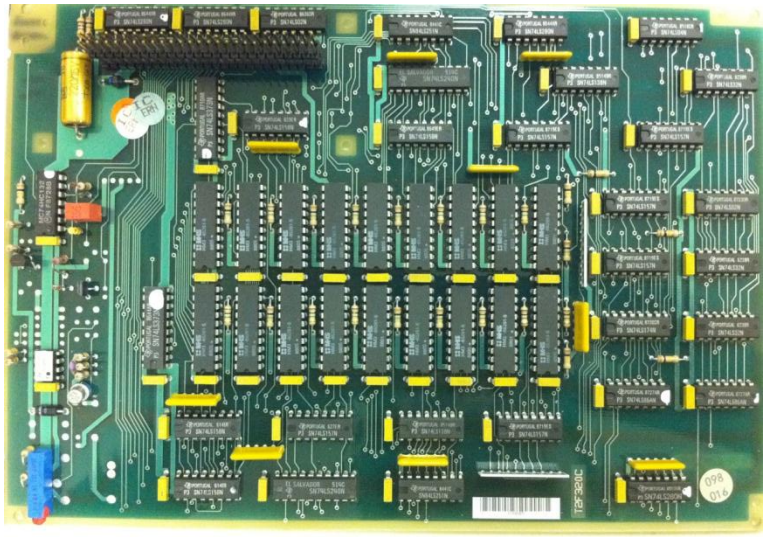
  Es. SN 7400 with 4 NAND gates

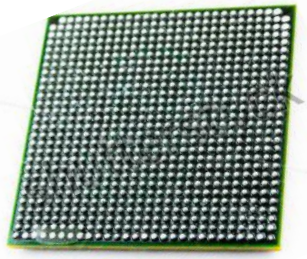




With this solution a simple clock needs a full board!

- In modern circuits all the logics in integrated in a single, programmable chip: a Programmable Logic Device (PLD)



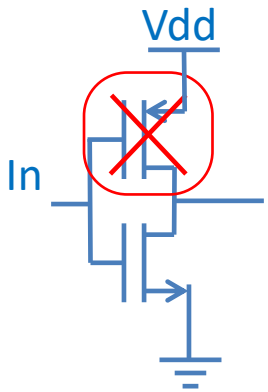


- A single PLD can easily integrate ~ 500.000 gates with 500-1000 pin !

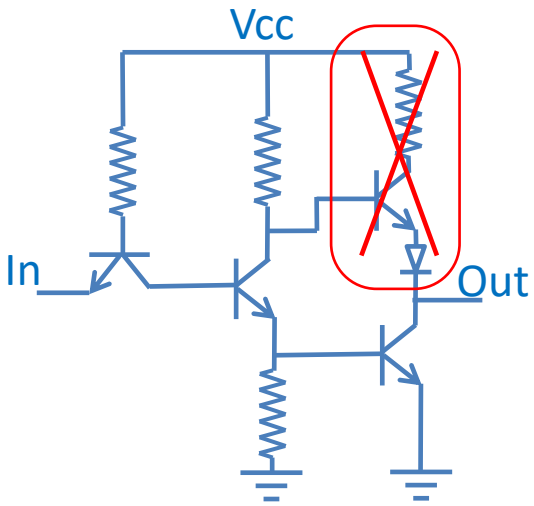
- Let's try and understand how to use it …

- Open Drain / Open Collector Gates -

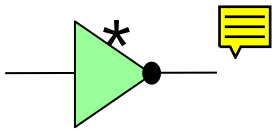
Open Drain NOT CMOS



Open Collector NOT TTL

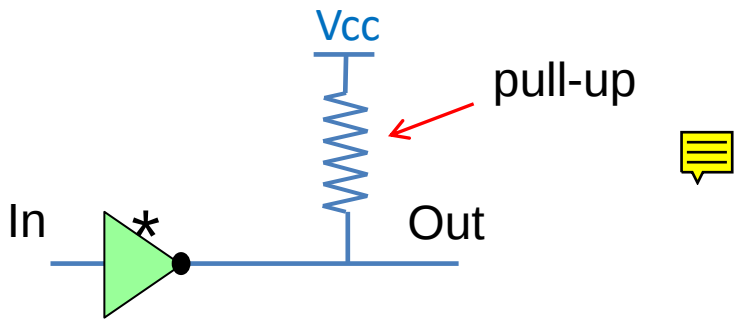


Open drain/collector gates are labelled with a star or similar symbol



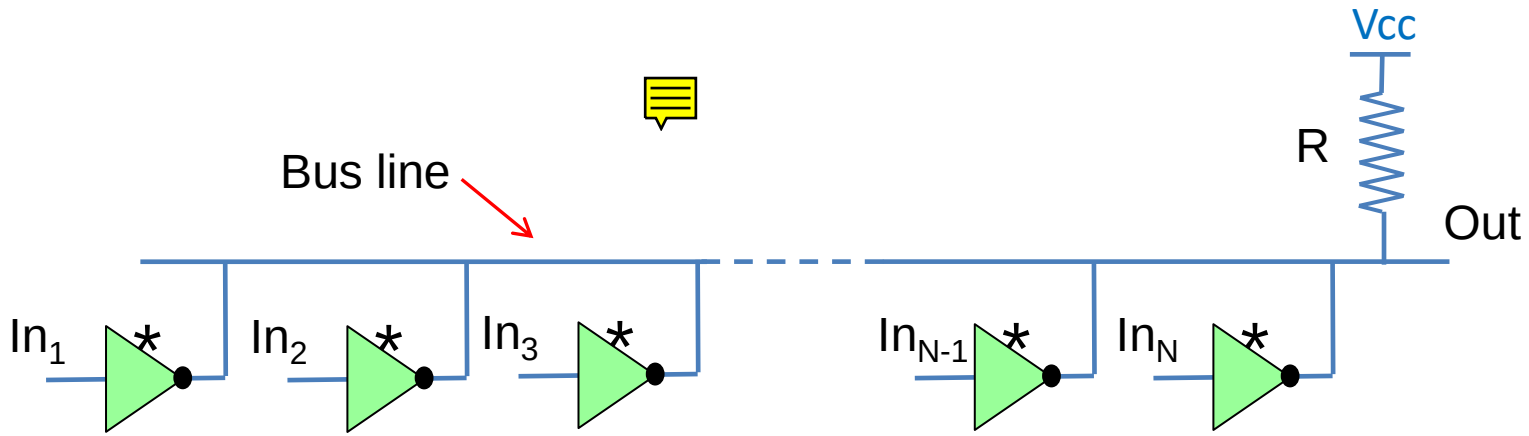
- The «open drain gates are obtained by removing the «high» MOS in CMOS and the «high» transistor-resistor-diode in TTL.
- The gate can only drive low, i.e. sink current from the output to ground, but can't source current from power to the output.
- Note: A Open Drain CMOS NOT gate can be realized with a single NMOS transistor

- Open Drain / Open Collector Gates -



- The gate needs a pull-up to drive the high level
- When Out = '0' the gate shorts the output to ground => 0V in Out
- When Out = '1' the gate is disconnected => Out = Vcc thanks to the pull-up

- Open Drain / Open Collector Gates -

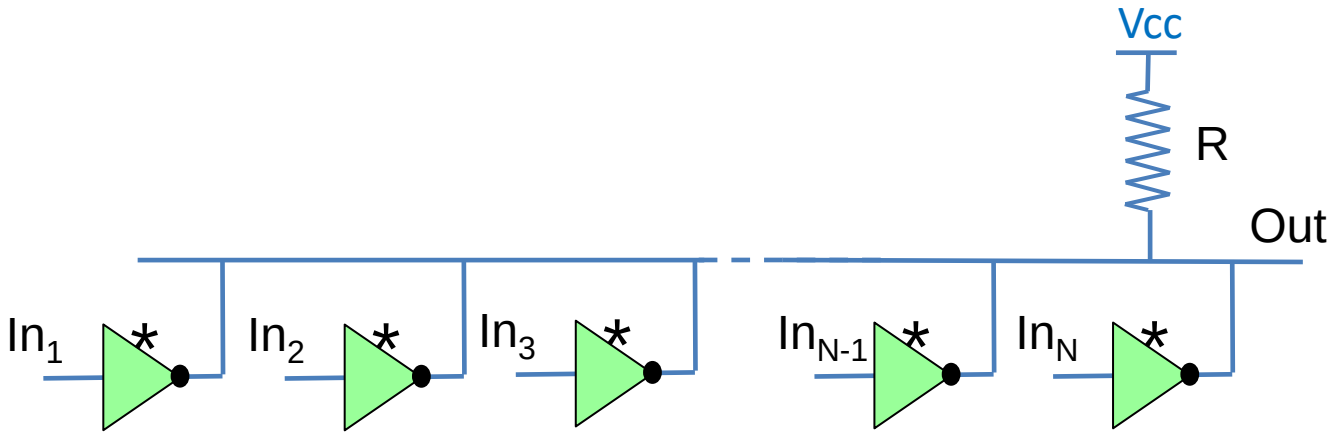


- The output of several gates can be connected on the same bus without risk of bus contention and circuit damage[1]
- In fact, each output or sinks current or is disconnected. The current is sourced to the bus only through the resistor R. => No risk of short

[1]see Elettronica dei Sistemi Digitali course

- Wired Logic -

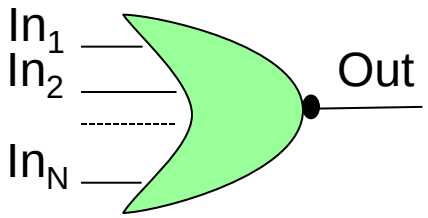
- Open Drain/Collector gates can be used for wired logic



NOR

In_1	In_2	,,,	In_N	Out
0	0	,,,	0	1
0	0	,,,	1	0
0	1	,,,	0	0
0	1	,,,	1	0
1	0	,,,	0	0
1	0	,,,	1	0
1	1	,,,	0	0
1	1	,,,	1	0

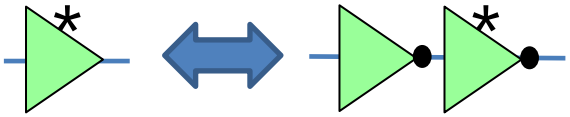
Out is HIGH only if ALL of the N In are LOW (Output disconnected). When one or more In is high, the corresponding output sinks current to ground and the Out goes LOW. This is a NOR gate with N inputs.



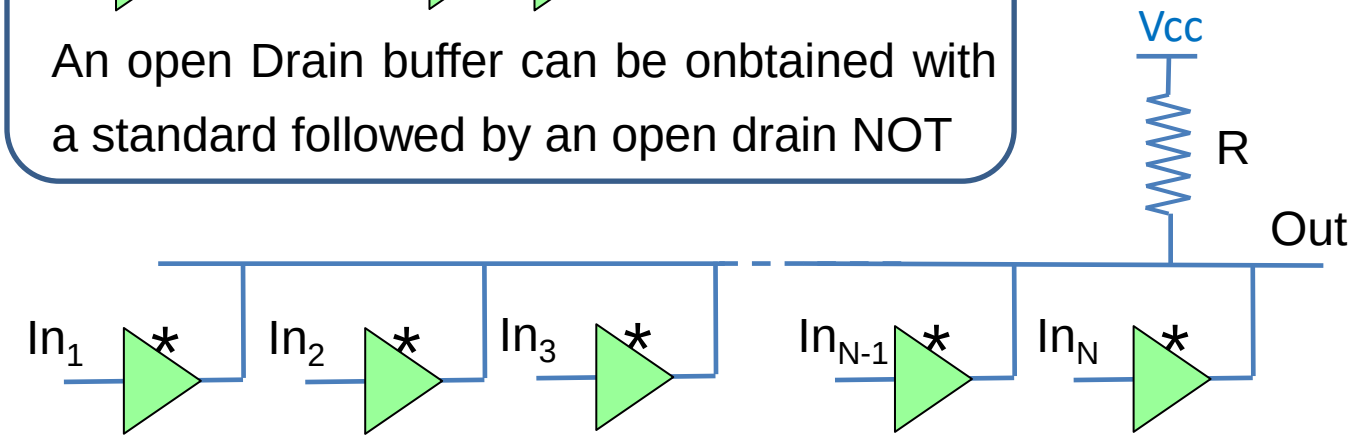
- Wired logic represents a simple way of realizing gates with several inputs

- Wired Logic -

- If we use a buffer instead of a NOT we have a wired AND



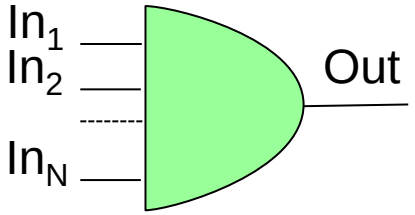
An open Drain buffer can be onbtained with a standard followed by an open drain NOT



AND

In_1	In_2	...	In_N	Out
0	0	...	0	0
0	0	...	1	0
0	1	...	0	0
0	1	...	1	0
1	0	...	0	0
1	0	...	1	0
1	1	...	0	0
1	1	...	1	1

Out is HIGH only if ALL of the N In are HIGH (Principal Mosfet Off). This is a AND gate with N inputs

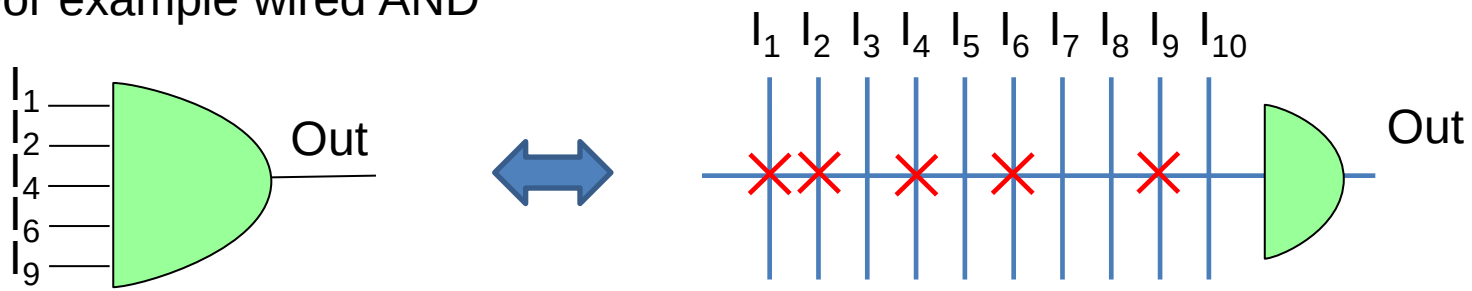


- Programmable Wired Logic -

- Towards a programmable logic…

  A wired logic can be represented as:

For example wired AND

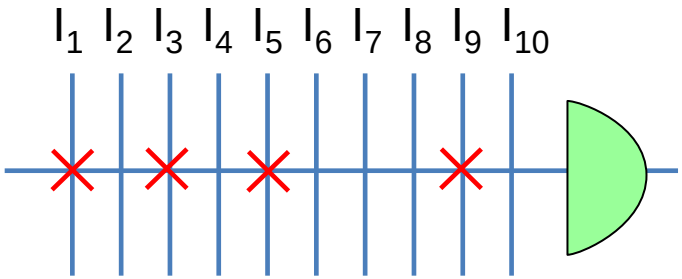


$Out = I_1 \bullet I_2 \bullet I_4 \bullet I_6 \bullet I_9$

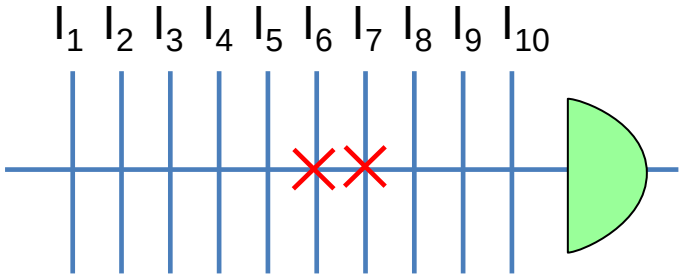
The possible inputs are reported as crossing lines. The inputs actually connected are labelled with a cross. Programming the AND means changing the actual connected inputs (crosses) among the possible inputs $I_{1\text{-}10}$

- Programmable Wired Logic -

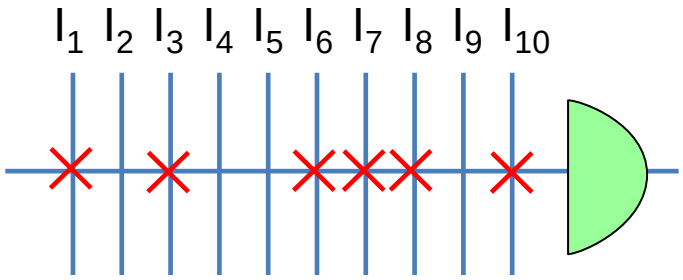
- For example…



$Out = I_1 \bullet I_3 \bullet I_5 \bullet I_9$



$Out = I_6 \bullet I_7$

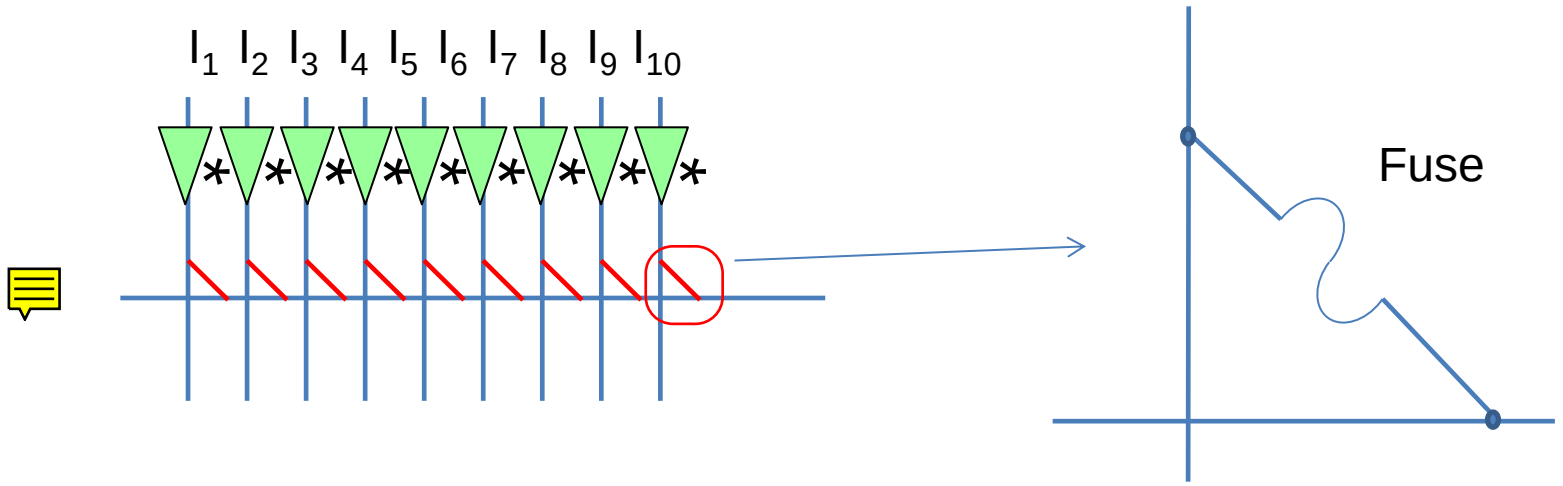


$Out = I_1 \bullet I_3 \bullet I_6 \bullet I_7 \bullet I_8 \bullet I_{10}$

How can this programmability be obtained in a real circuit?

- Programmable Wired Logic -

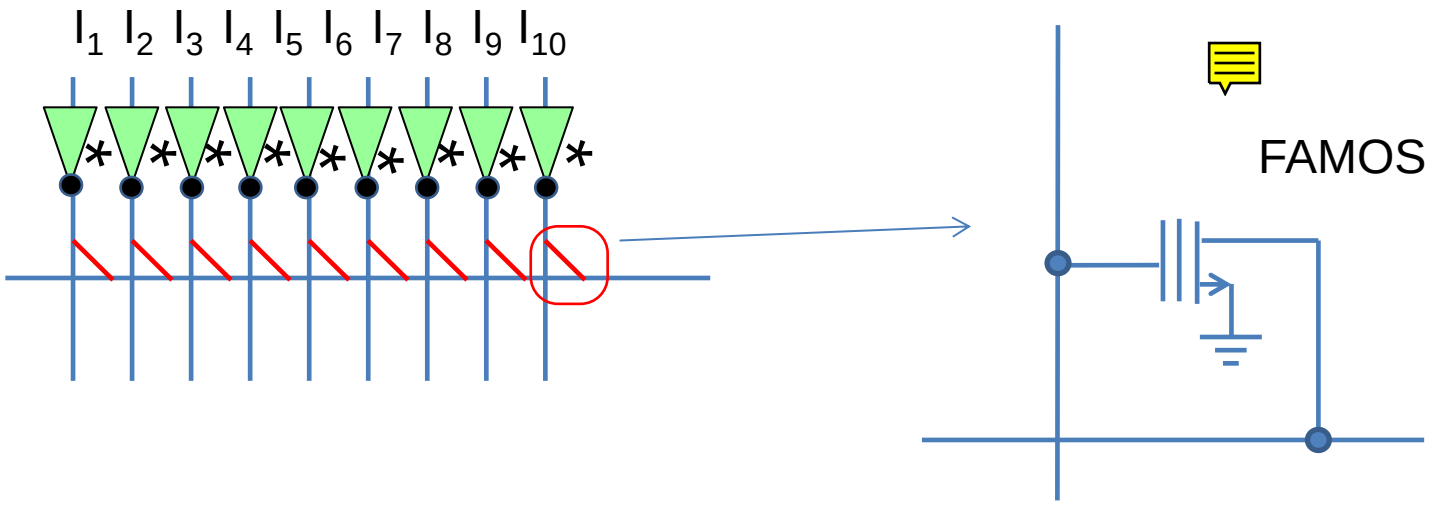
- …By using matallic fuses



A «fuse» is made by a thin metal strip. A high current is used to blow the fuse and remove the connection. Once the connection is removed it can not be restored. These devices can be programmed once (OTP)

- Programmable Wired Logic -

- …By using the Floating gate Avalance injection MOS (FAMOS)

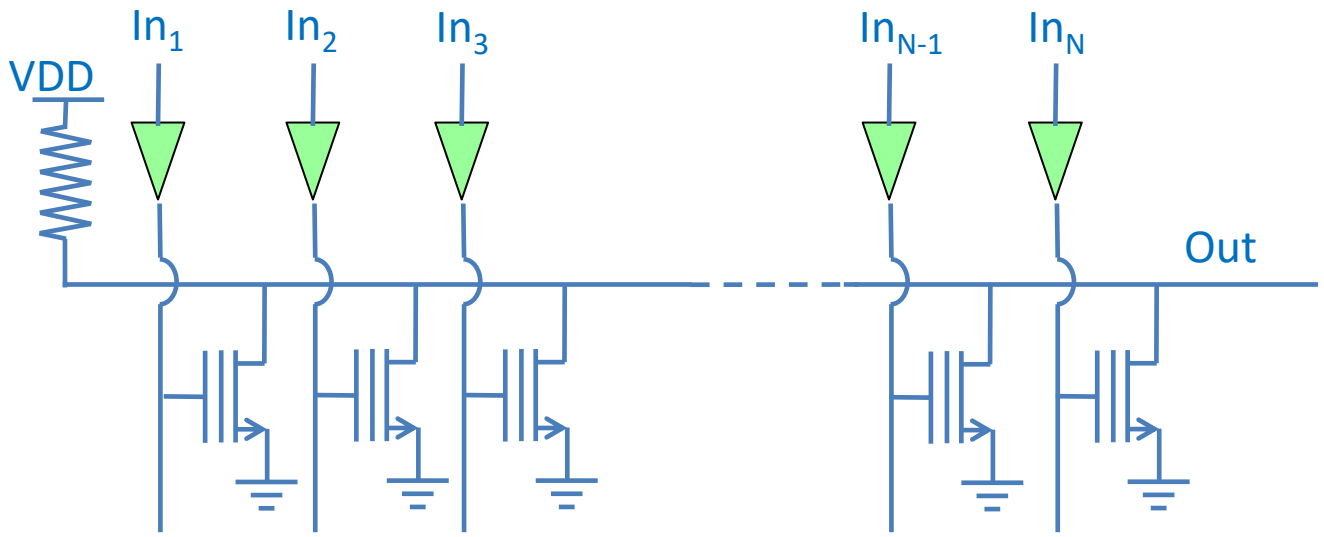


The FAMOS is used also in EEPROM and can be re-programmed several times. When electrons are stored in the floating gate, the FAMOS is an open circuit, when electrons are removed from the floating gate, the FAMOS works like a standard MOS.

Note that in this case we use standard input buffers (NOT in this case), since each NOT is connected just to one FAMOS gate

- Programmable Wired Logic -

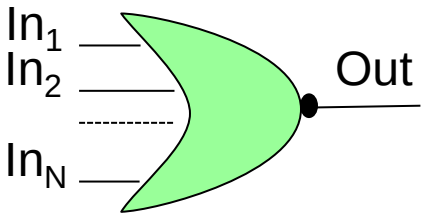
- Programmable NOR with FAMOS



NOR

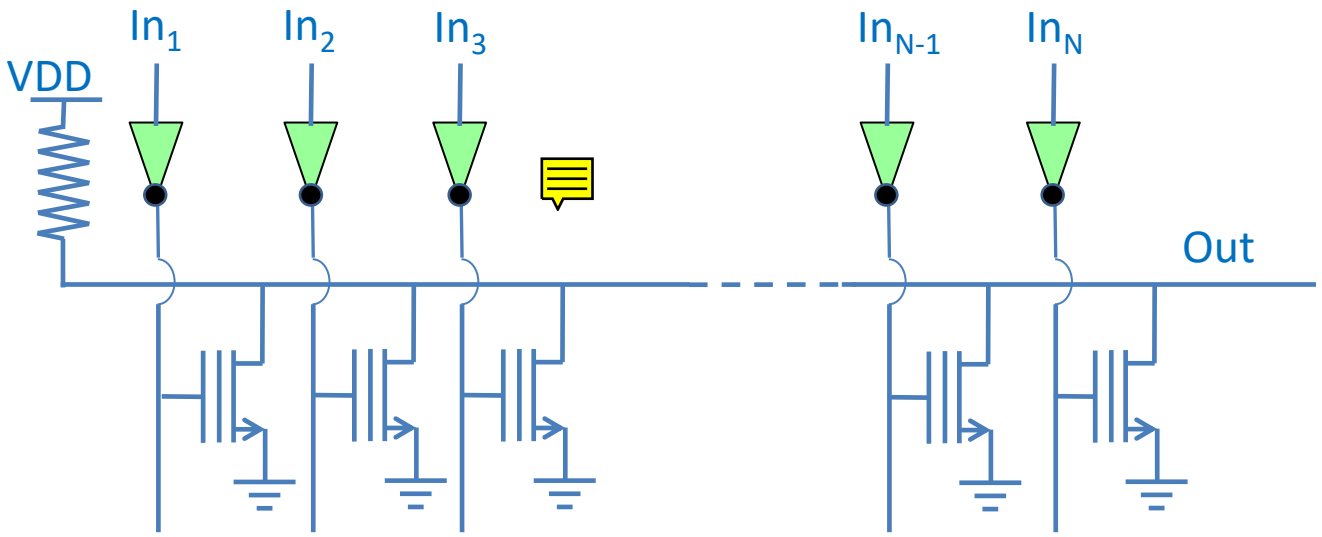
In_1	In_2	,,,	In_N	Out
0	0	,,,	0	1
0	0	,,,	1	0
0	1	,,,	0	0
0	1	,,,	1	0
1	0	,,,	0	0
1	0	,,,	1	0
1	1	,,,	0	0
1	1	,,,	1	0

Out is HIGH only if ALL of the N In are LOW (FAMOS Off). When one or more In is high, the corresponding FAMOS is switched ON, and the Out goes LOW. This is a NOR gate with N inputs. Of course, disabled FAMOS (with electrons on floating gate) are always OFF and the corresponding input does not matter



- Wired Logic -

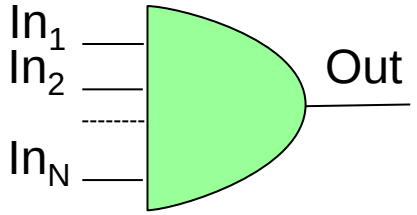
- Programmable AND with FAMOS



AND

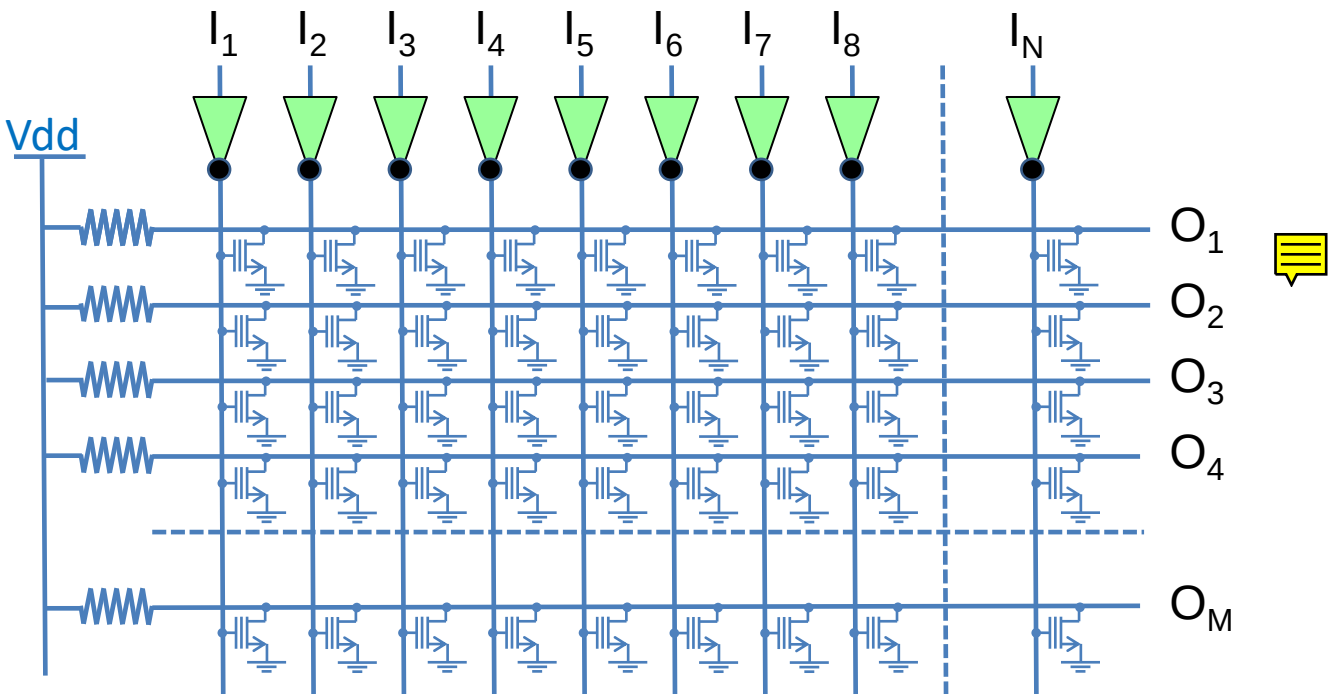
In_1	In_2	...	In_N	Out
0	0	...	0	0
0	0	...	1	0
0	1	...	0	0
0	1	...	1	0
1	0	...	0	0
1	0	...	1	0
1	1	...	0	0
1	1	...	1	1

If a NOT is used as input buffer, we obtain a programmable AND instead of NOR



- Programmable AND Matrixes-

- We can produce a matrix of wired AND as follows

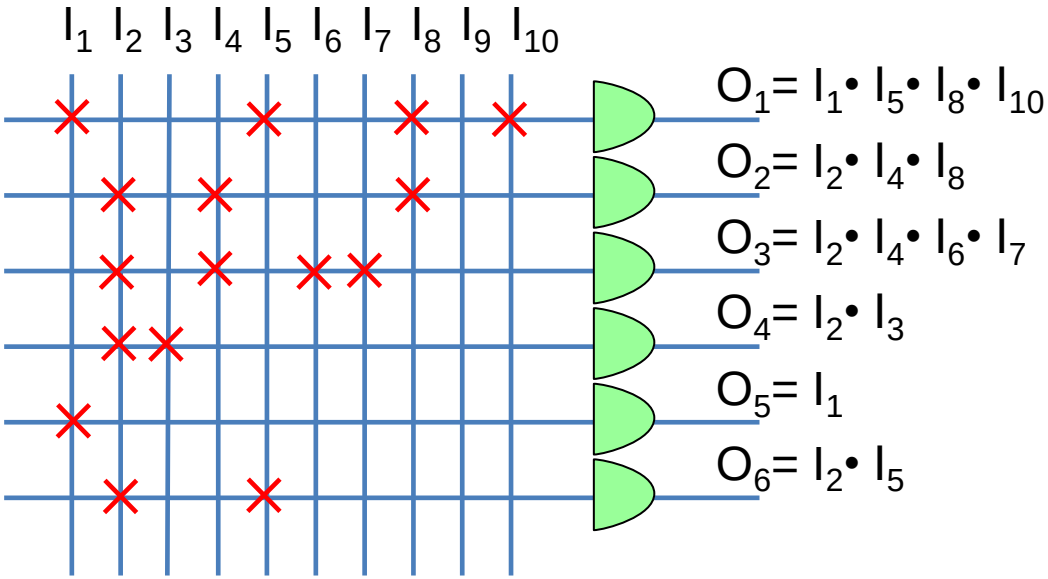


In this example we have a N input – M output programmable AND matrix

The FAMOSes select which inputs are connected to which outputs

- Programmable AND Matrix-

- The matrix is usually represented as :

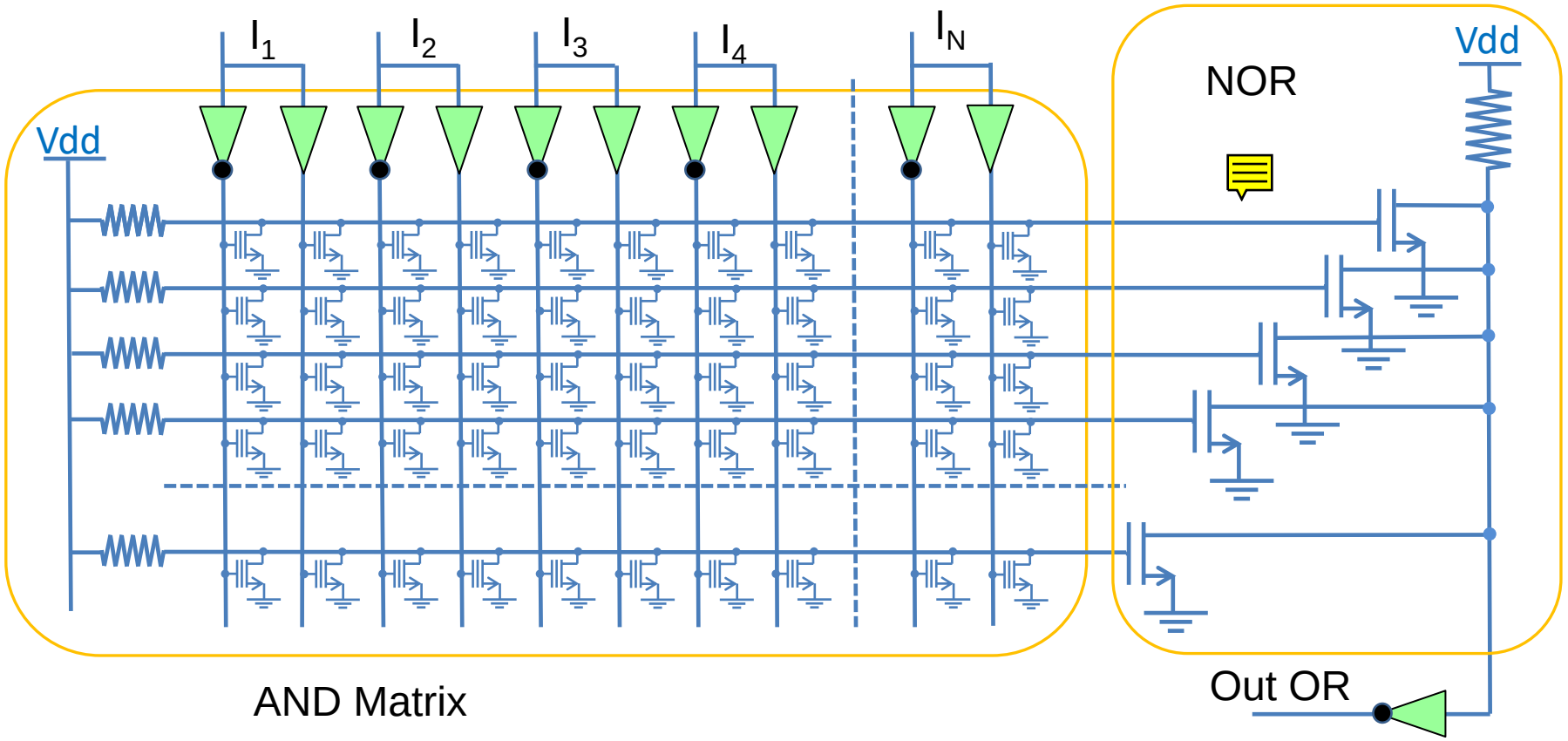


In this example we have 6 wired AND with 10 possible inputs each

Each cross represents a «working» FAMOS, the missing cross a «isolating» FAMOS

- PAL: Programmable Array Logic -

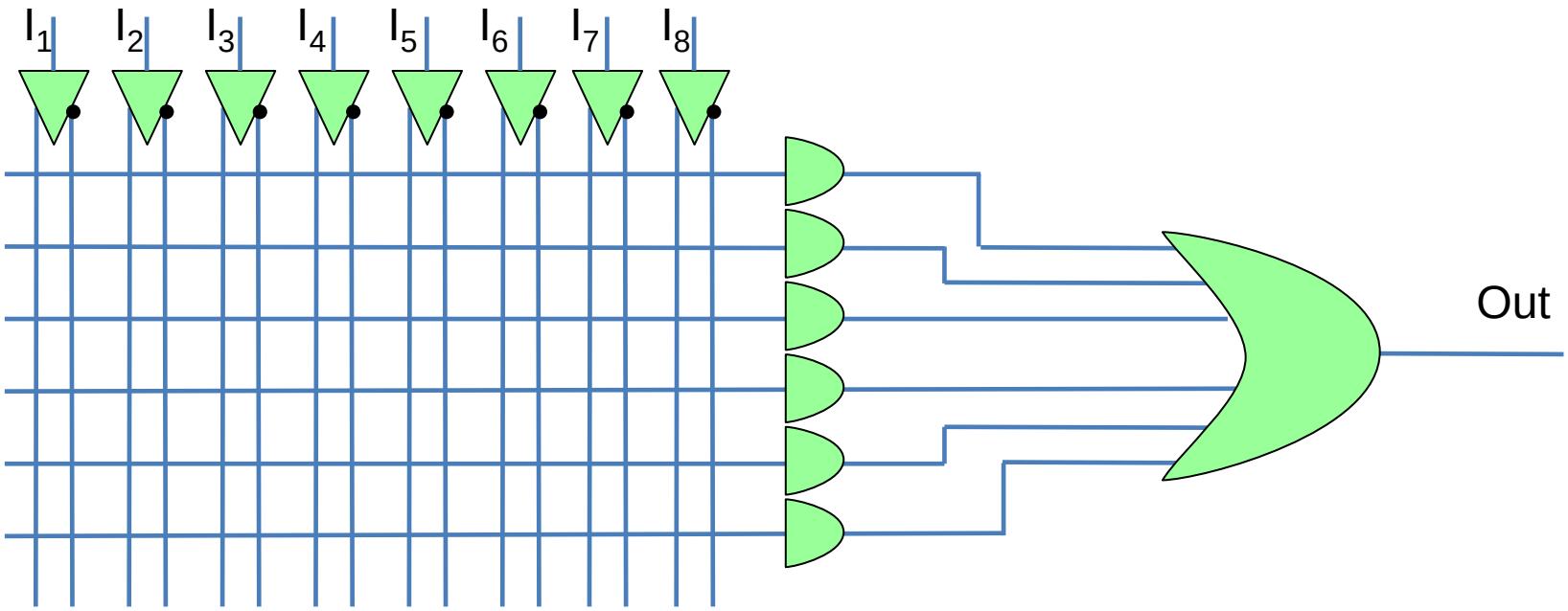
- PLA is realized by a programmable AND matrix and a fixed OR



- The inputs feed the matrix in direct and negate form

- PAL: Programmable Array Logic -

- The PAL is usually represented as :



In this example 8 input feed a programmable AND matrix followed by a 6 input OR

- PAL: Programmable Array Logic -

- Every logic function has a AND-OR representation (2-level).

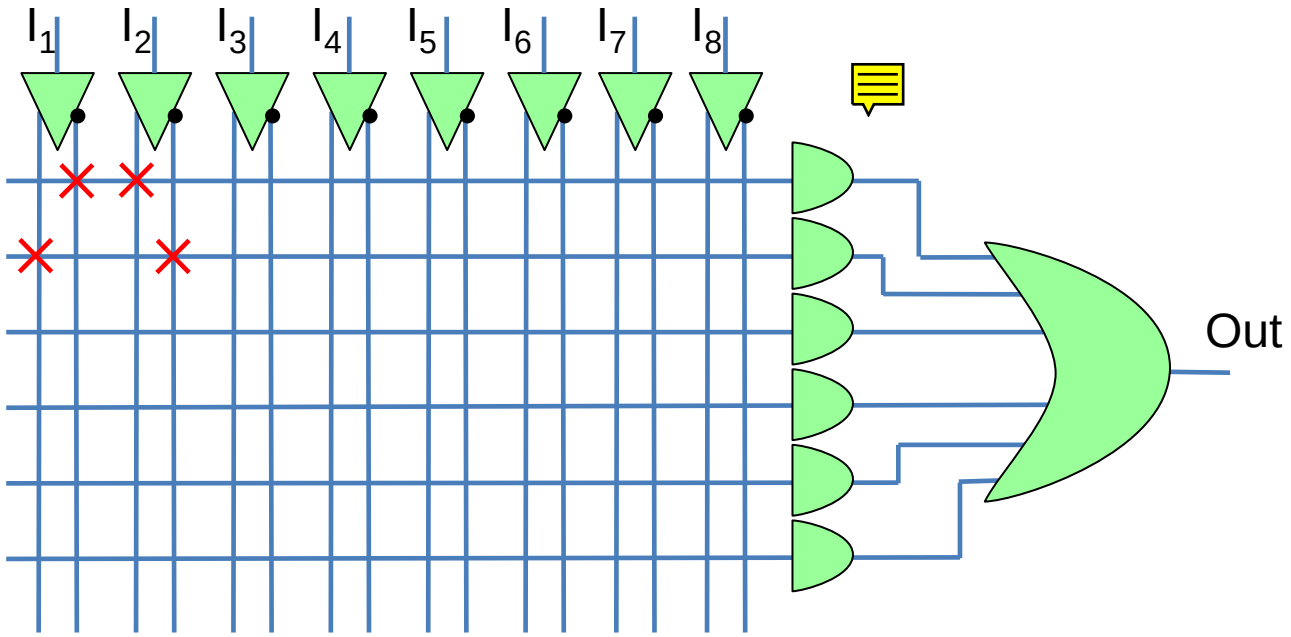
Every logic function can be programmed in a PAL

(provided that the PAL has enough resources)

Example:

I_1	I_2	Out
0	0	0
0	1	1
1	0	1
1	1	0

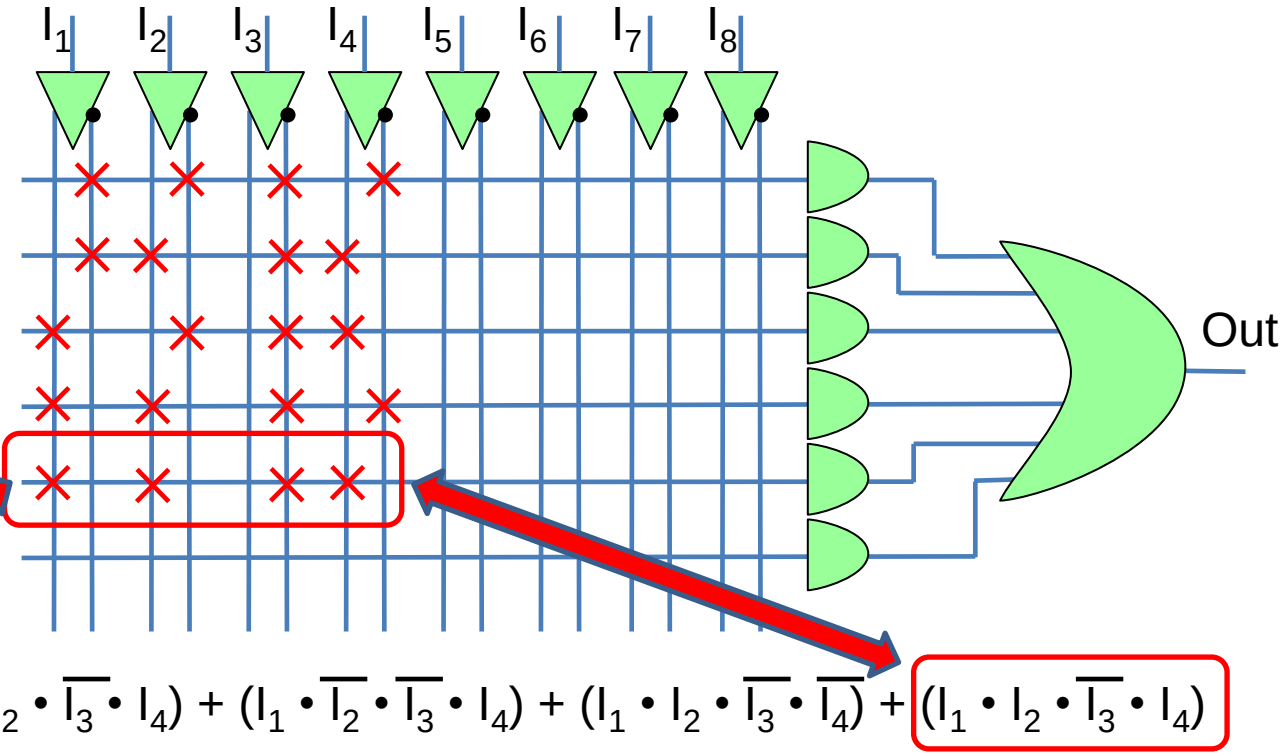
$$Out = (\overline{I_1} \cdot I_2)+(I_1 \cdot \overline{I_2})$$



- PAL: Programmable Array Logic -

Example:

I_1	I_2	I_3	I_4	Out
0	0	0	0	1
0	1	0	1	1
1	0	0	1	1
1	1	0	0	1
1	1	0	1	1
All other combinations				0



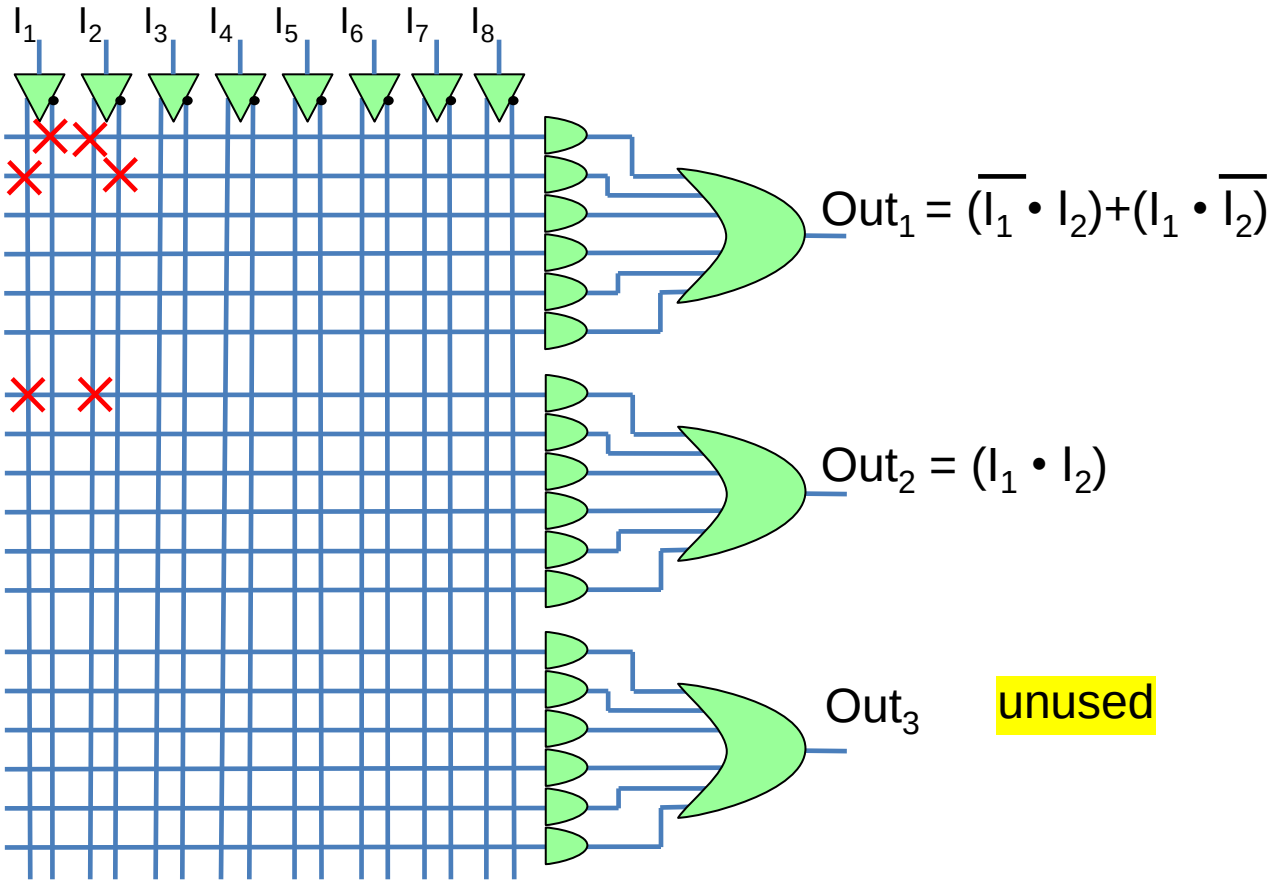
$$Out = (\overline{I_1} \cdot \overline{I_2} \cdot \overline{I_3} \cdot \overline{I_4}) + (\overline{I_1} \cdot I_2 \cdot \overline{I_3} \cdot I_4) + (I_1 \cdot \overline{I_2} \cdot \overline{I_3} \cdot I_4) + (I_1 \cdot I_2 \cdot \overline{I_3} \cdot \overline{I_4}) + (I_1 \cdot I_2 \cdot \overline{I_3} \cdot I_4)$$

Every product term (truth table line) is realized by a wired AND in the PAL

A function with N inputs and M product terms needs a PAL with, at least, N inputs and M lines towards the OR

- PAL: Programmable Array Logic -

- The PAL includes several output (OR gates)



unused

Example:

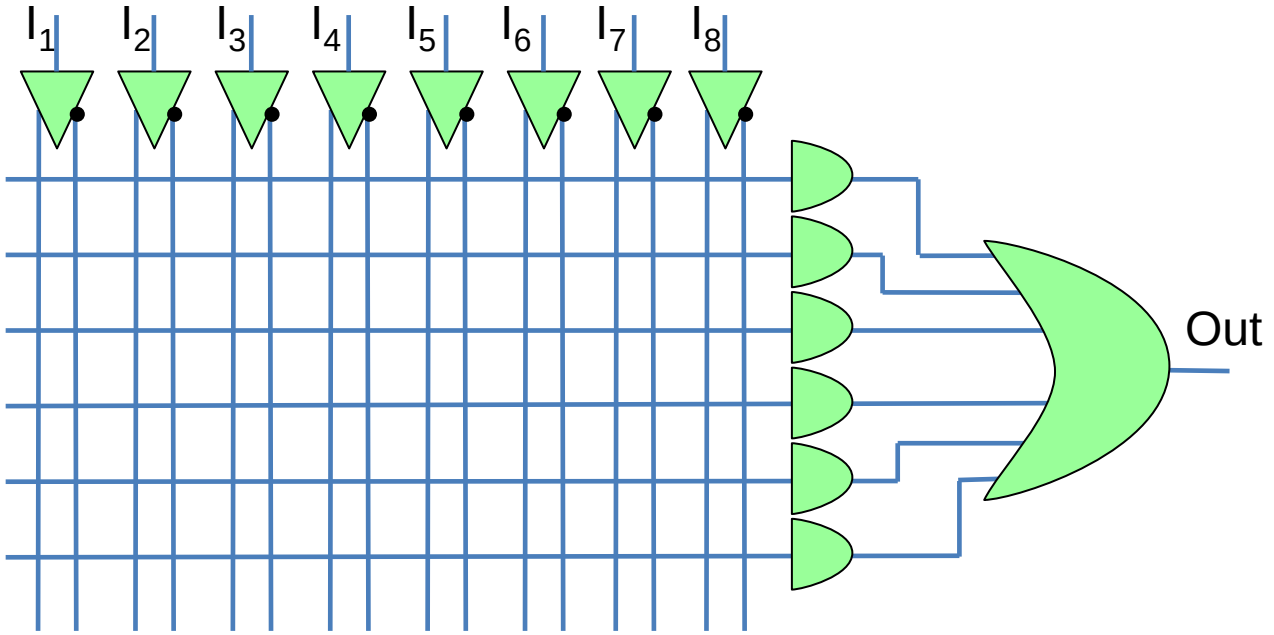
I_1	I_2	Out_1	Out_2
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

The inputs (matrix columns) are common to all outputs

- PAL: Programmable Array Logic -

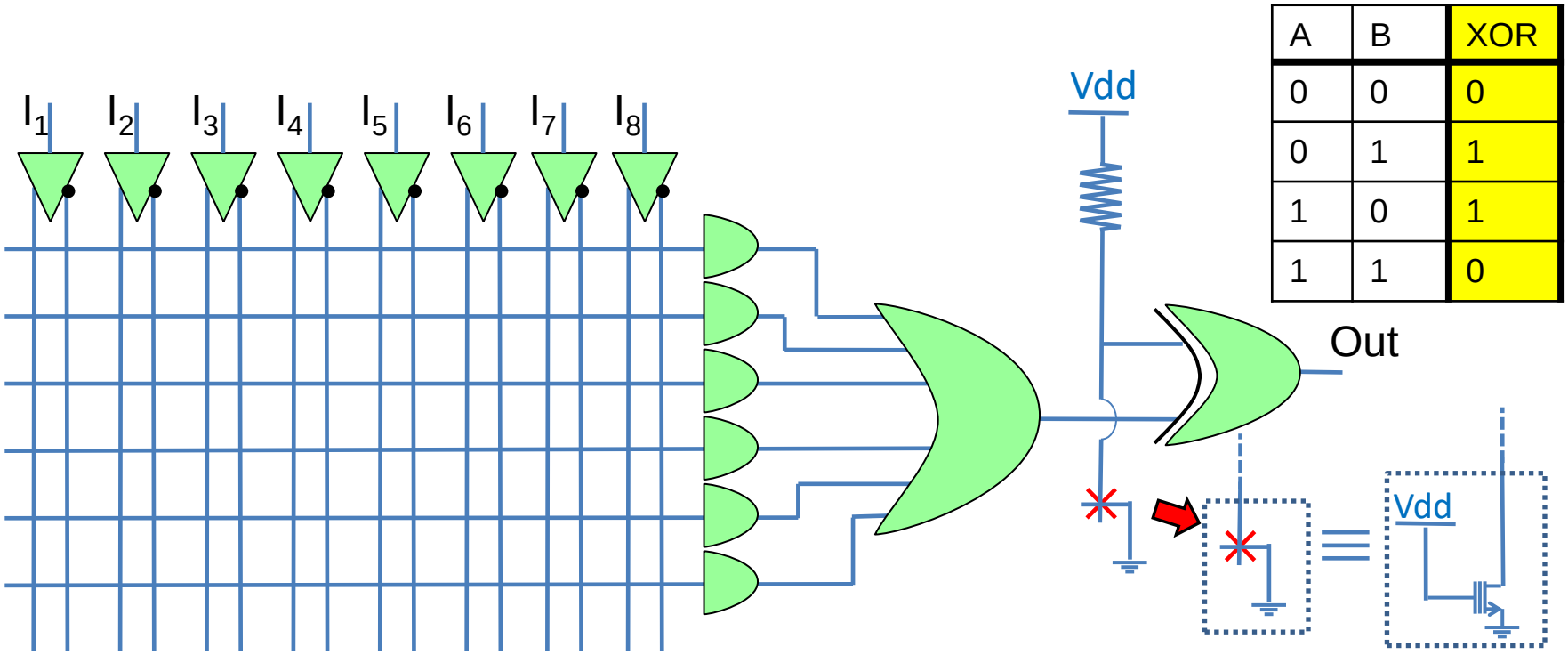
Example:

I_3	I_2	I_1	Out
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0



The ‘out’ of this logic funcion has seven ‘1’. As it is, it needs 7 product terms, but this PAL has only 6. This function cannot be realized. However it has only a ‘0’ in output, so $\overline{Out} = I_3 \bullet I_2 \bullet I_1$. The neg function would be very easy to realize….

- PAL: Improvements: Additional XOR -



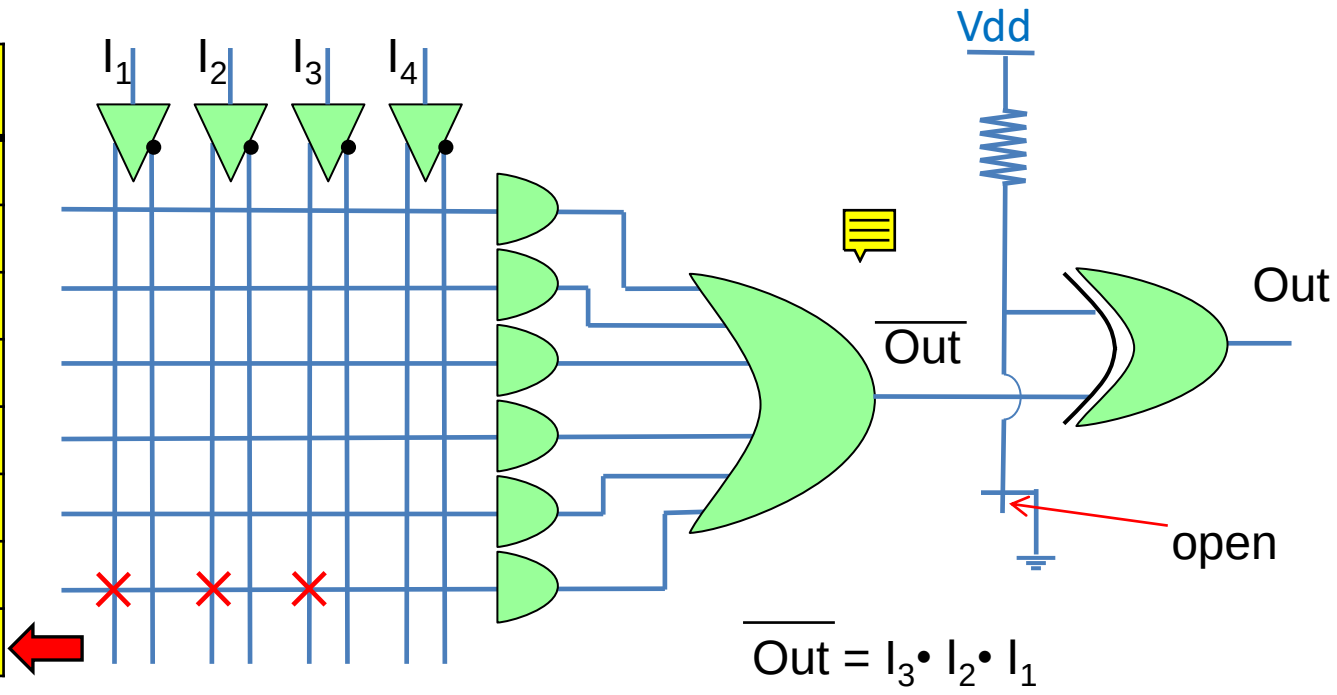
A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

The XOR is sort of «programmable NOT». If one input is ‘1’ the other is iverted, if it is ‘0’ the other is not inverted. With a FAMOS we can decide if inverting or not the output. This is useful for the synthesis of function with more ‘1’ than ‘0’ in the output

- PAL: Improvements: Additional XOR -

Example:

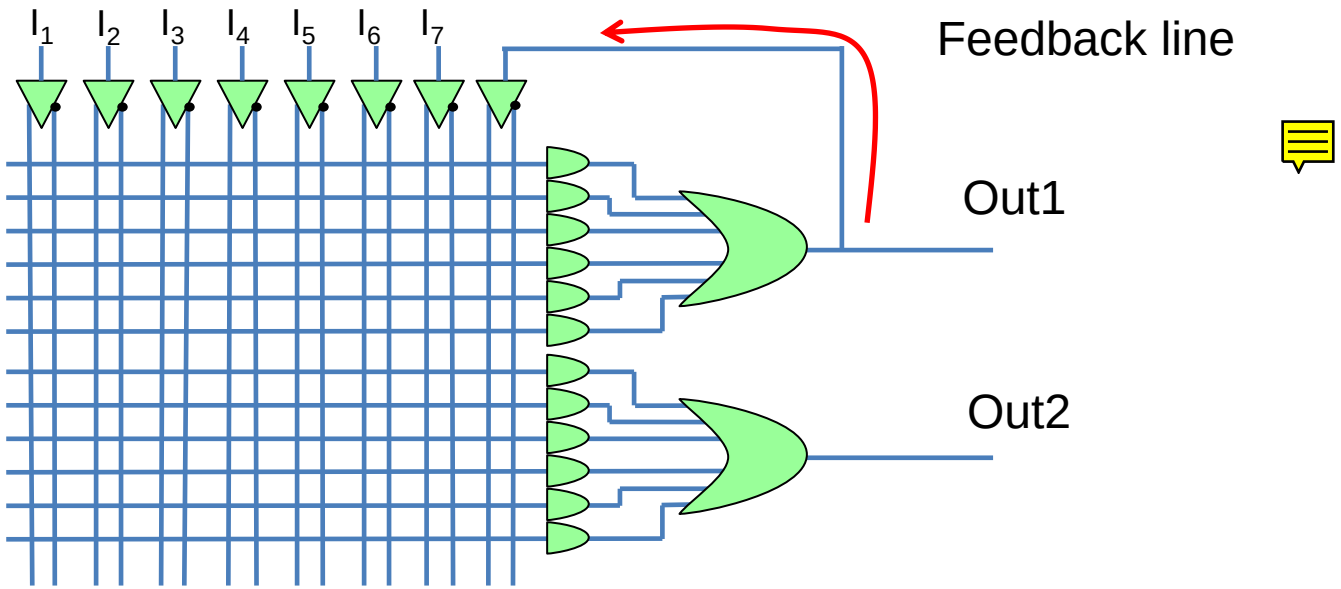
I_3	I_2	I_1	Out	$\overline{Out}$
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	1	0
1	0	1	1	0
1	1	0	1	0
1	1	1	0	1



In the AND matrix the $\overline{Out}$ is coded, then the final XOR inverts the $\overline{Out}$.

The programmable XOR allows to synthesize the ‘1s’ or the ‘0s’ of a logic function, whichever are more convenient

- PAL: Improvements: feedback line -



It is possible to feed an output back in the AND matrix. This allows the realization of more complex functions

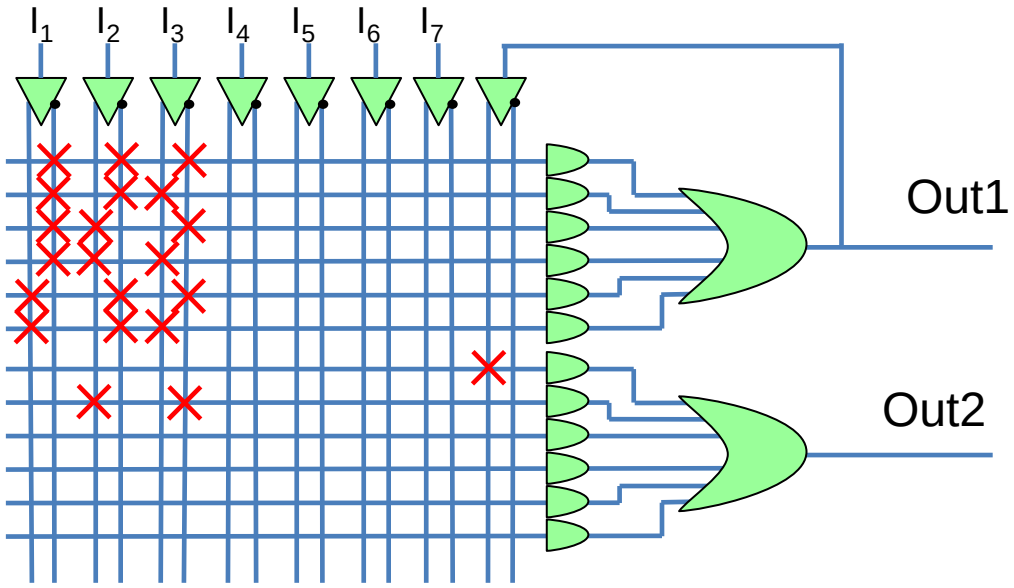
- PAL: Improvements: feedback line -

Example:

I_3	I_2	I_1	Out
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Out1

Out2



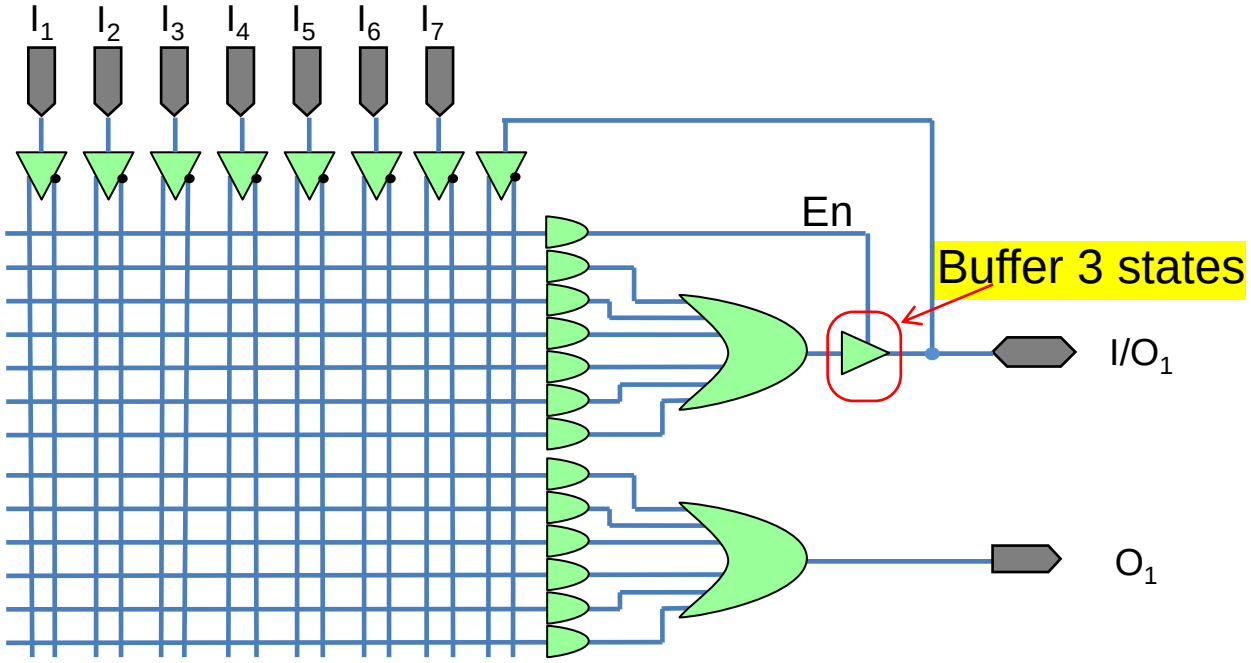
$$Out2 = Out1 + I_3 \bullet I_2 \bullet \overline{I_1}$$

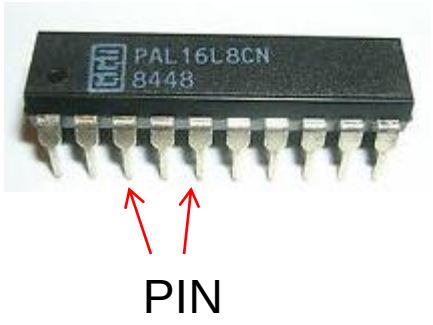
Out1 synthesizes the first six ‘1s’ of the table. Out1 is routed back in the AND matrix and added to the last ‘1’.

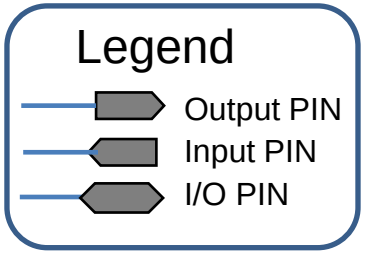
This is a 4-level function implementation => more delay

- PAL: towards commercial devices -

- Programmable Input/Output (I/O)





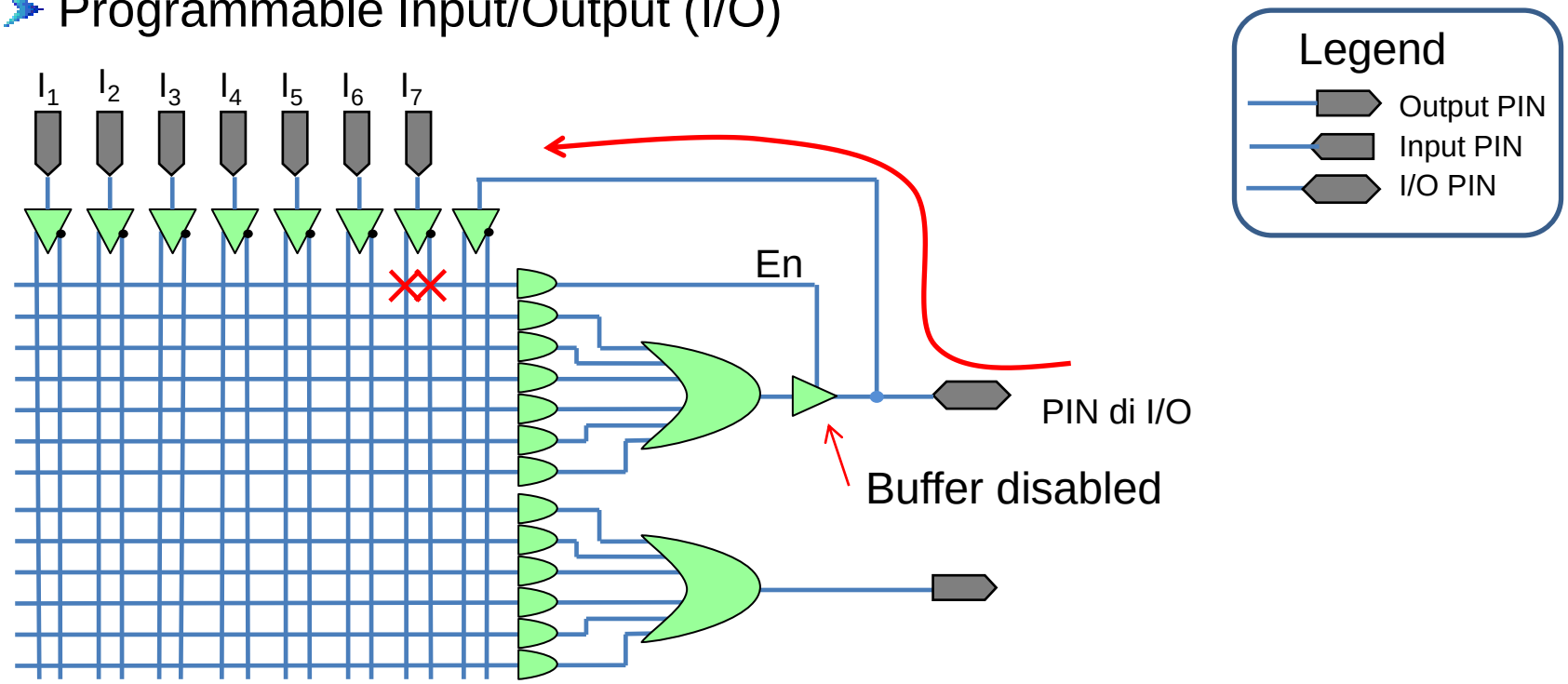


The external connections of a devices can be dedicated inputs (e.g. $I_1 – I_7$), dedicated outputs (O_1), or programmable Input/Output (I/O_1).

When the buffer 3 state is active (En = 1) the pin is an output, otherwise (En = 0) it is an Input. The En signal can be programmed from the AND matrix

- PAL: Programmable Array Logic -

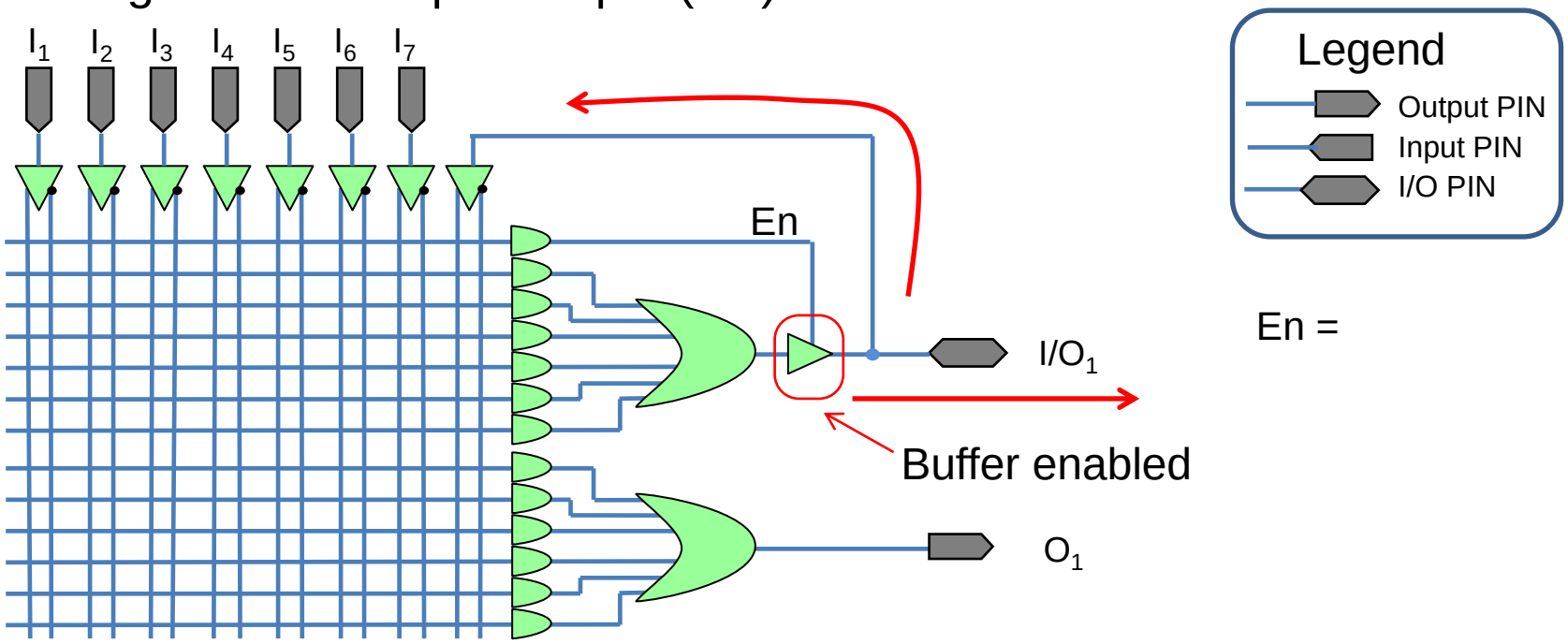
- Programmable Input/Output (I/O)



$I_7 \bullet \overline{I_7}$ = En = 0, thus the buffer is disabled and the PIN is an input, which feeds the AND matrix like the other inputs. In this case the logic connected to the disabled buffer can't be used and is wasted.

- PAL: Programmable Array Logic -

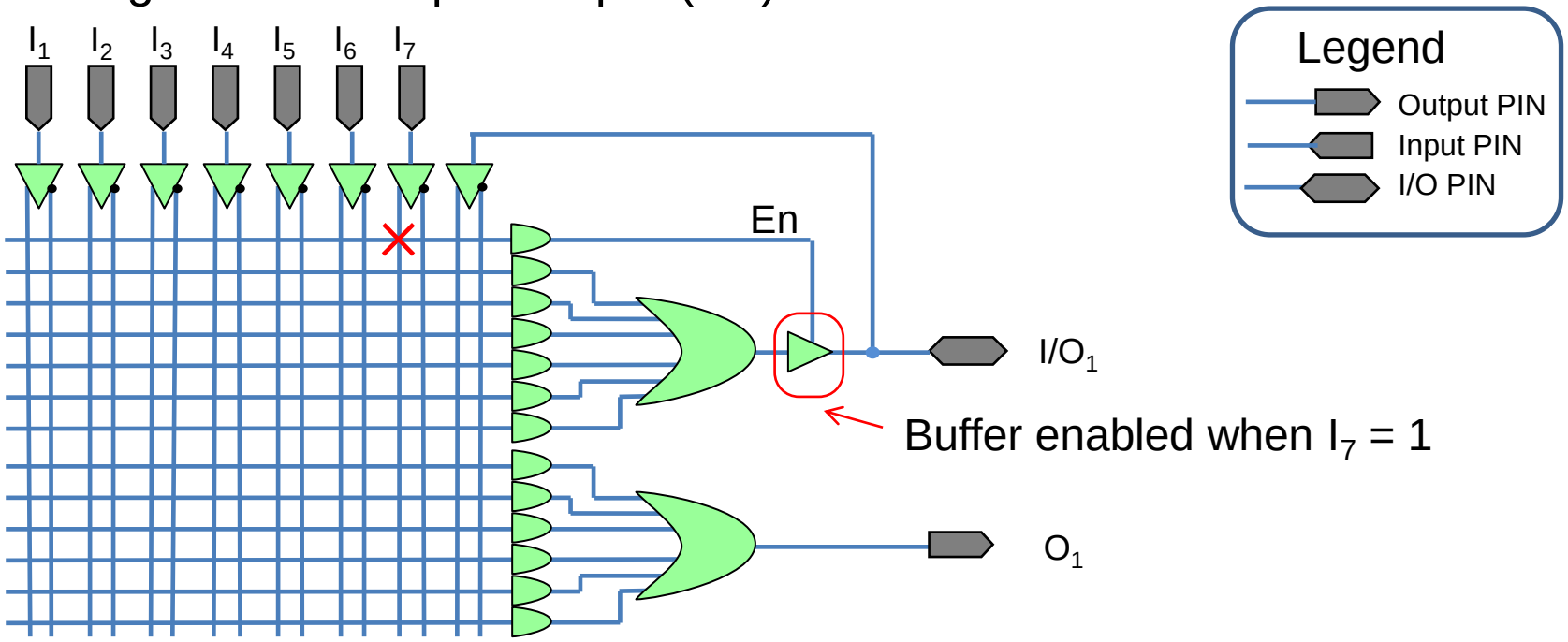
- Programmable Input/Output (I/O)



When the AND connected to En has not connections (no crosses) En = '1' (pull-up on the line) and the PIN is an output. Note that the output signal has a feedback into the AND matrix, and can be re-used in the logic

- PAL: Programmable Array Logic -

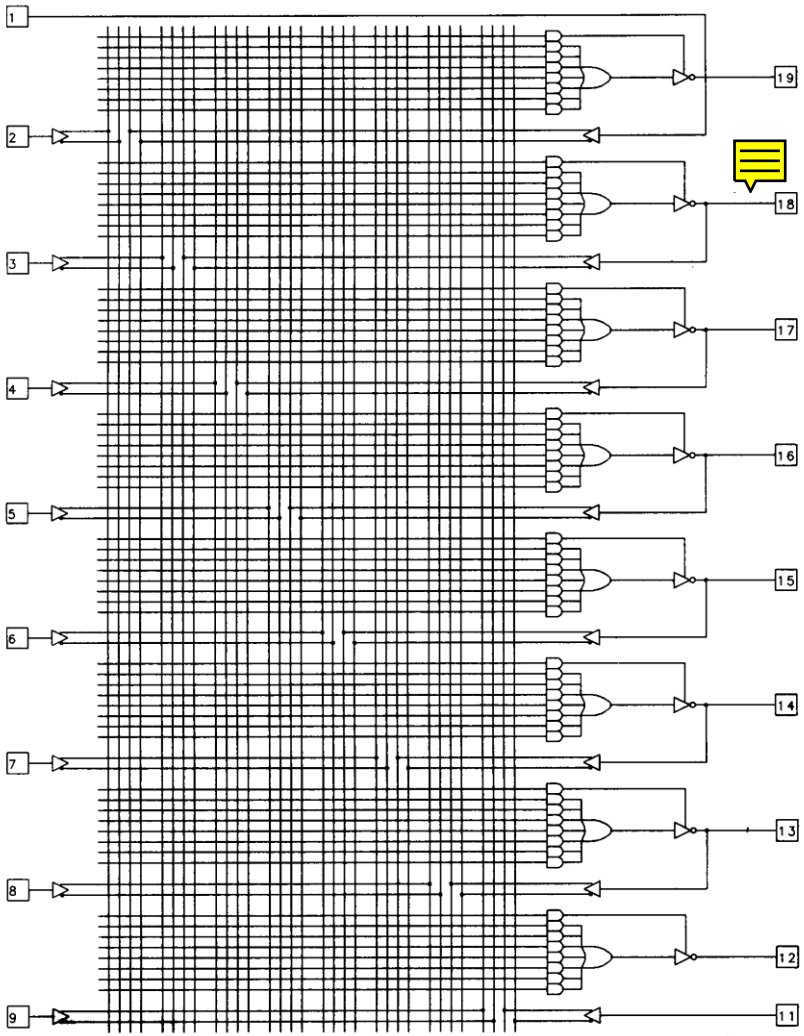
- Programmable Input/Output (I/O)



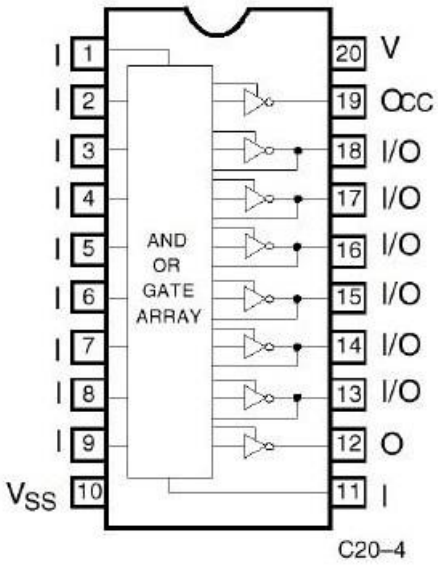
In this example En = I_7 and I_7 works as an output enable.

More complex configurations are possible with a suitable programming

- Commercial Example: PAL 16L8 -



PAL 16L8

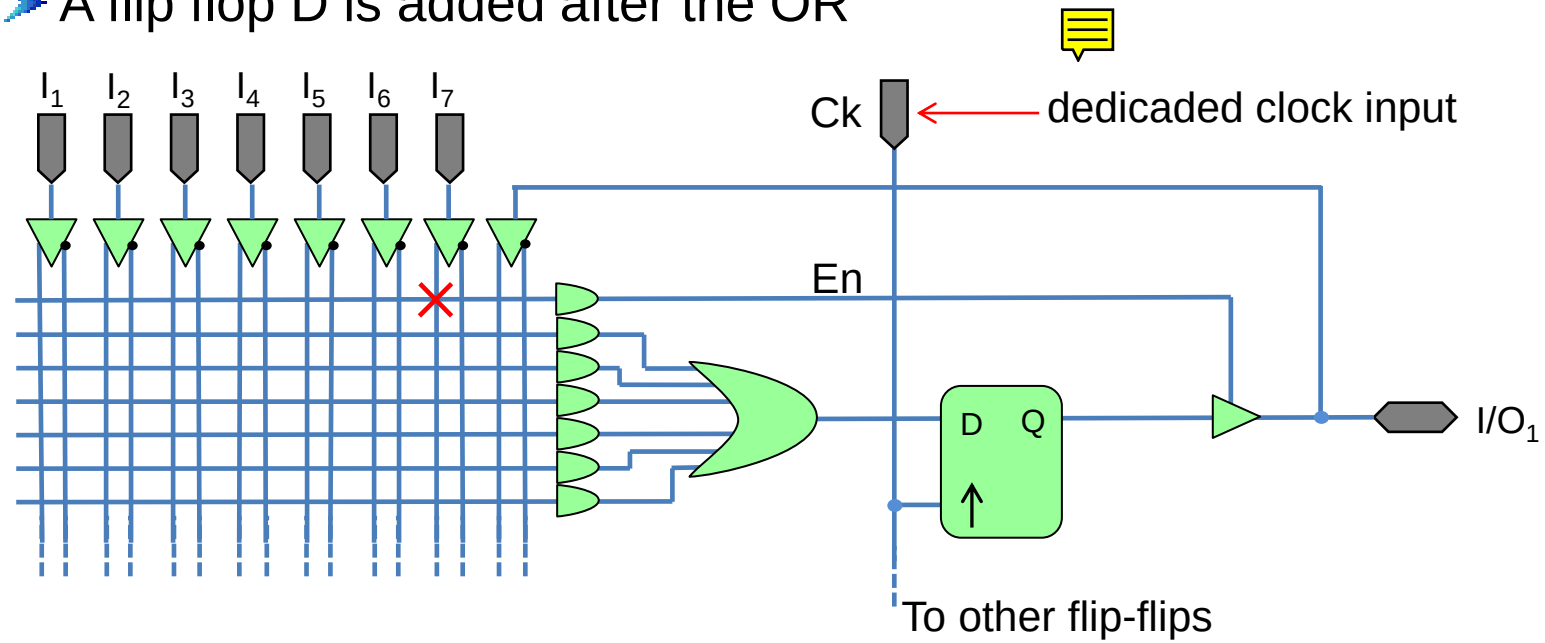


PAL with

- 10 dedicated input (I)
- 2 dedicated output (O) with 3 states
- 6 pin switchable between I or O (I/O)

- Registered Programmable Logic -

- A flip flop D is added after the OR

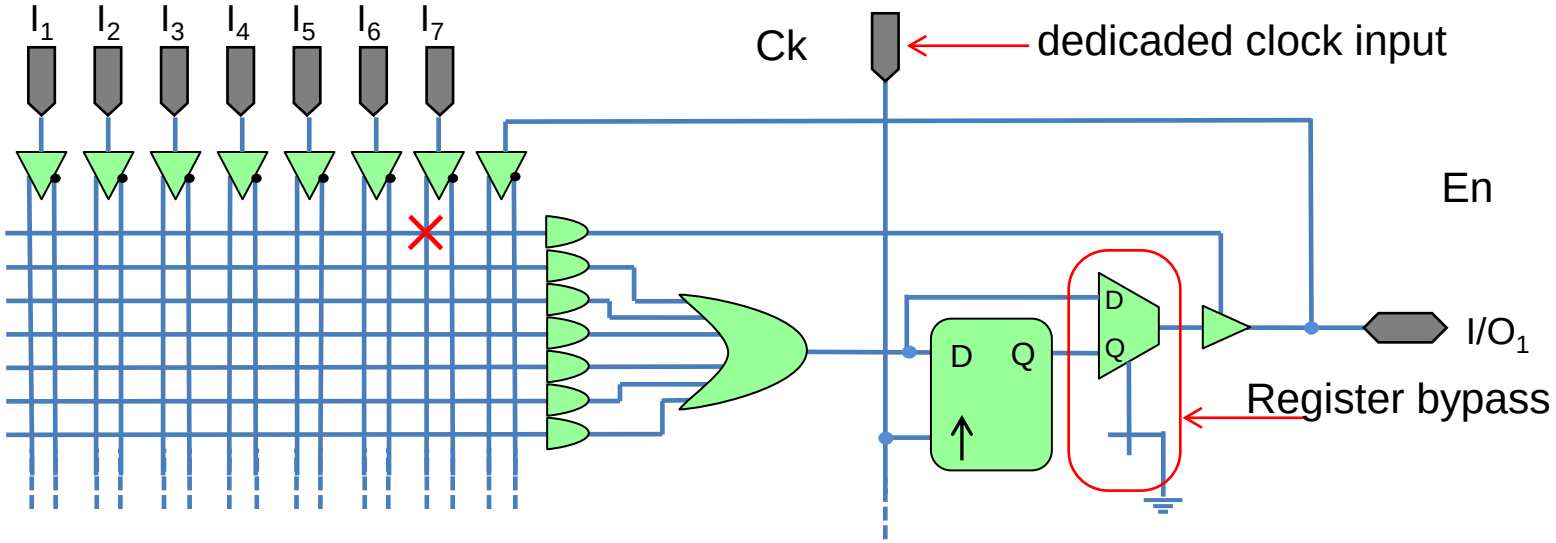


The clock is fed by a dedicaded input to all the flip flops of the devices

- Registered Programmable Logic -

- Register by-passas

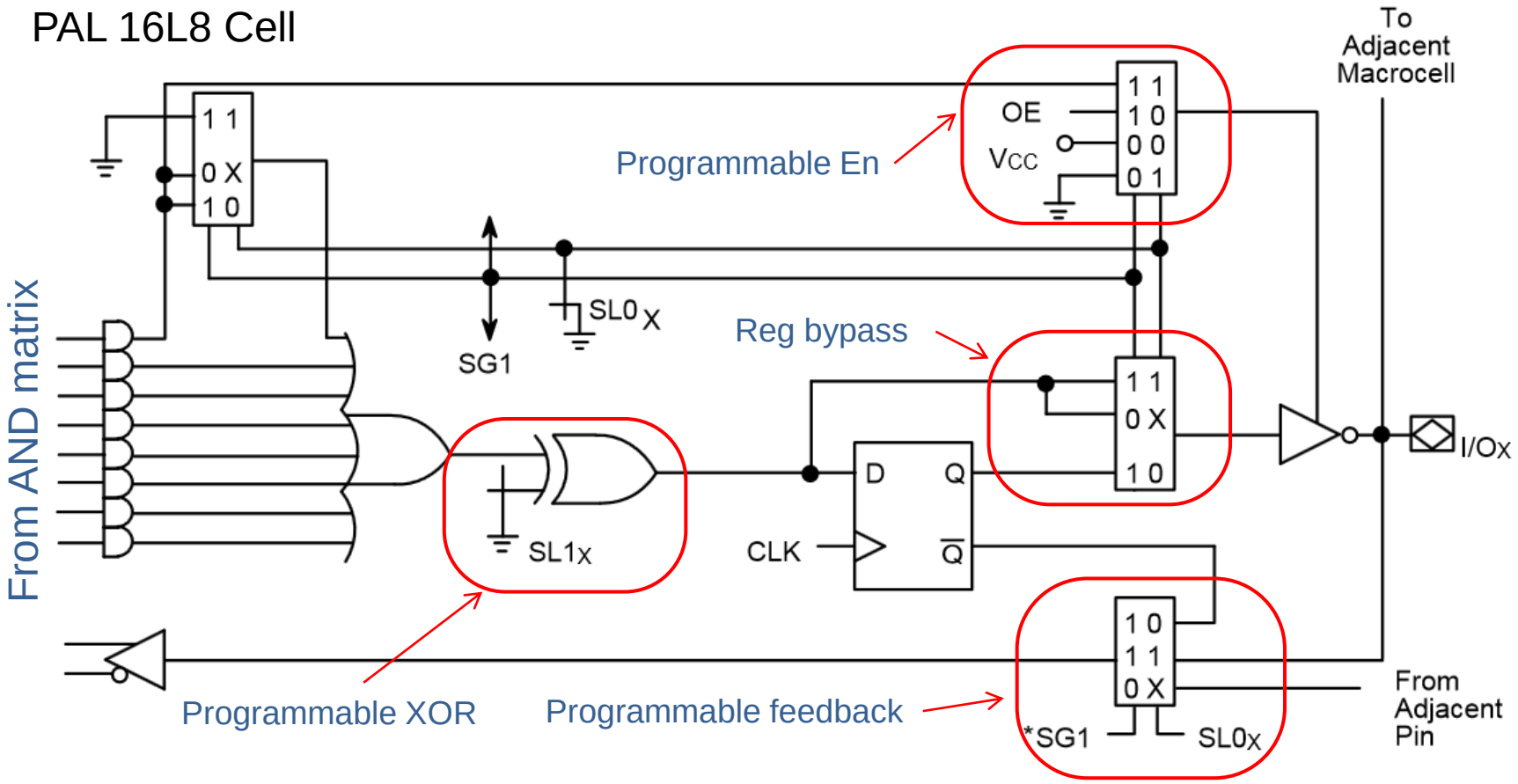
A programmable mux can be added to bypass (or not) the register



The mux can be programmed to select the output from the register (Q) or bypass the register (D).
The cell synthetizes sequential or combinatorial logic, depending on how the mux is programmed.

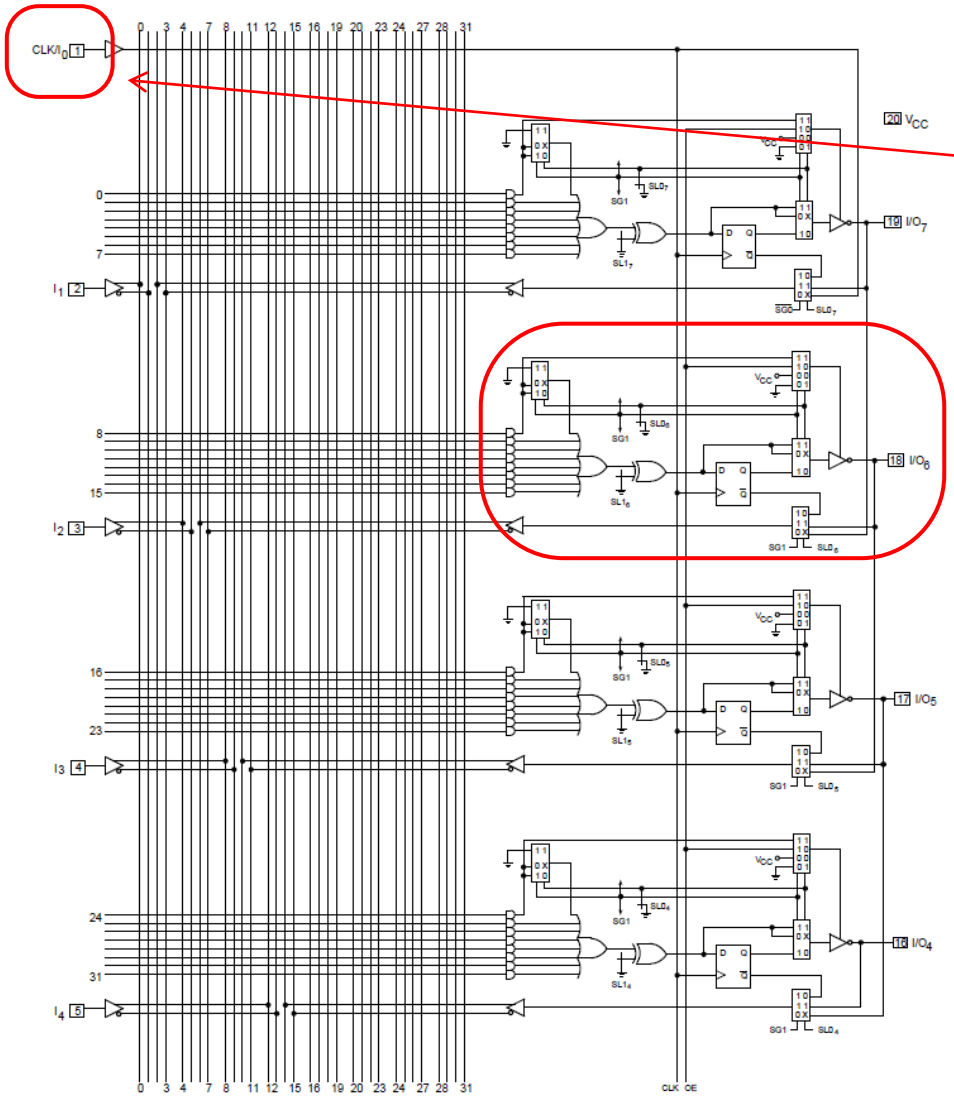
- Commercial Example: PAL 16V8 -

PAL 16L8 Cell



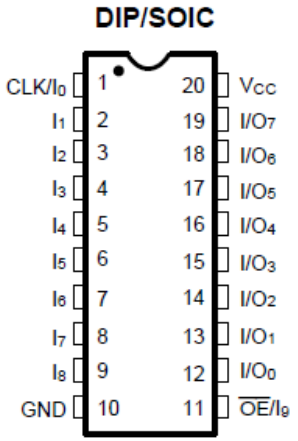
The behaviour of the cell is programmed by the SL0,SG1,SL1 bits

- Commercial Example: PAL 16V8 -



The clk is common to all of the registers

cell

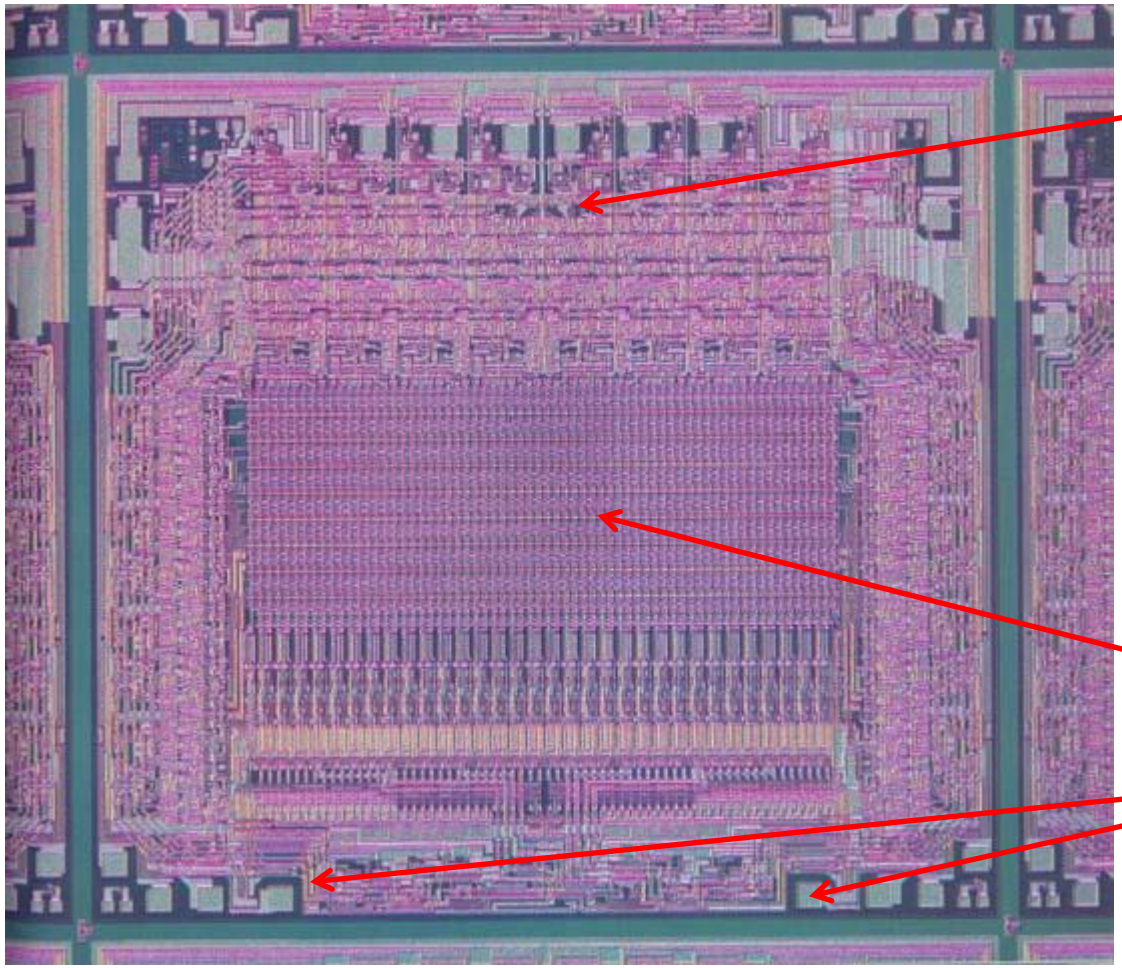


The PAL 16V8 has 8 cells

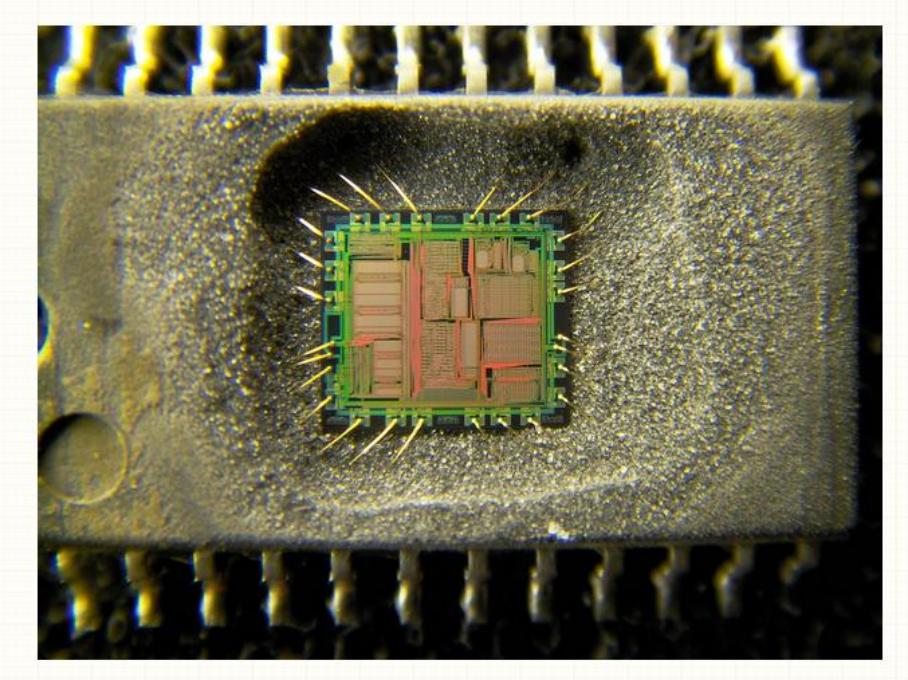
Other cells below…

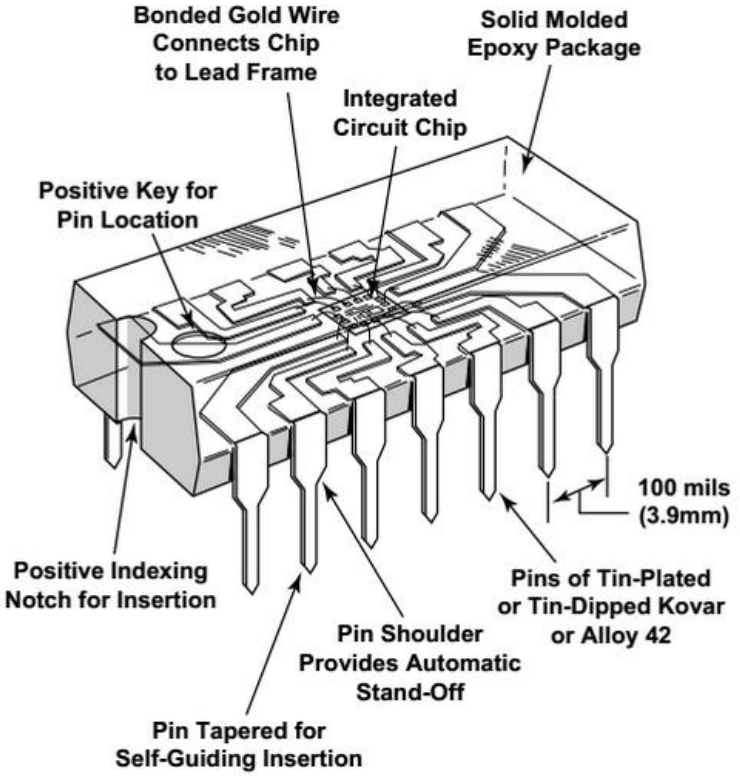
- Commercial Example -

DIE of PAL 16L8



- Commercial Example -



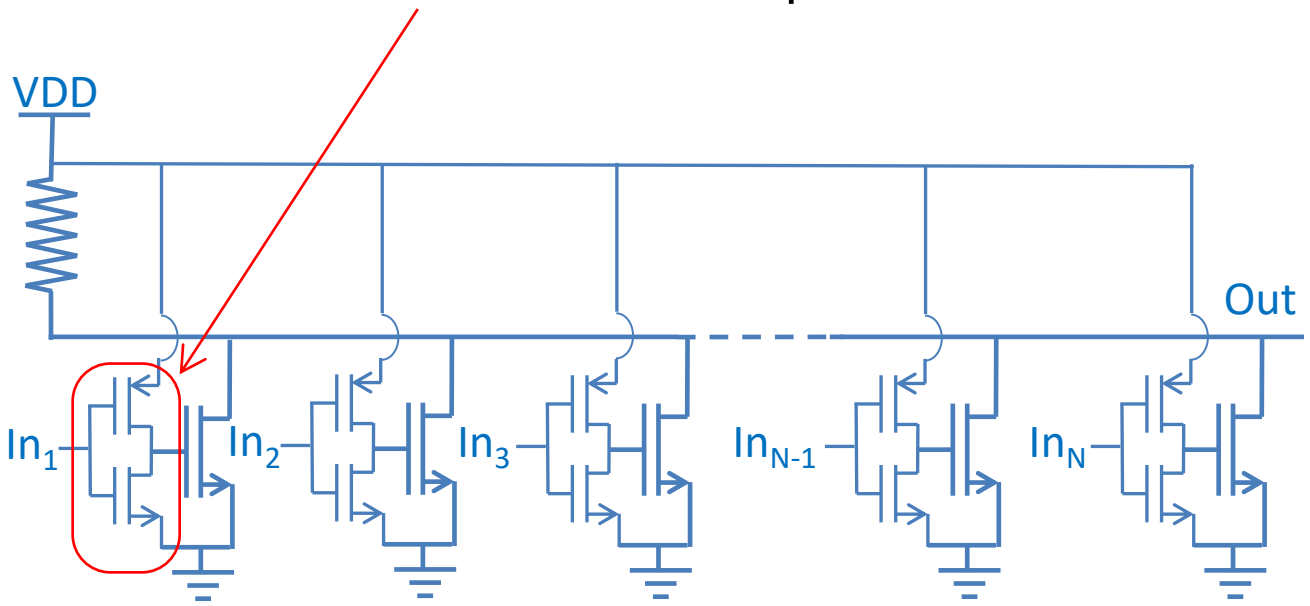


- Summary -

- Wired logic is useful to obtain logic functions with several inputs
- Fuses or FAMOS allows programmability
- Programmable AND matrix followed by fixed OR allows the realization of programmable 2-level logic functions
- Several improvements (feedback, XOR, etc.) can be added
- A register can be present for sequential logic implementation
- PAL devices are (were) commercially available which implement such functions

- Wired Logic -

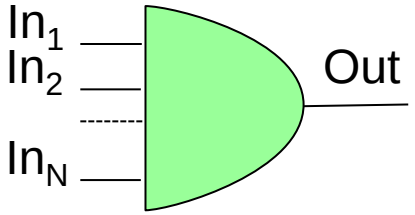
- If we add an inverter on each input we have a wired AND



AND

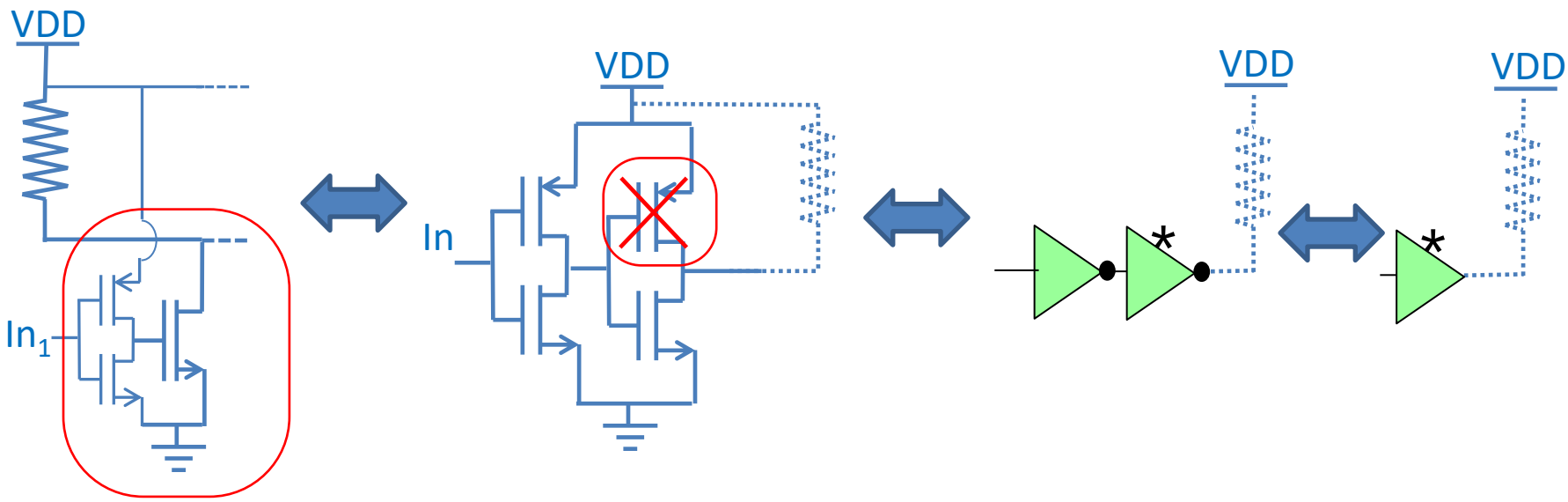
In_1	In_2	...	In_N	Out
0	0	...	0	0
0	0	...	1	0
0	1	...	0	0
0	1	...	1	0
1	0	...	0	0
1	0	...	1	0
1	1	...	0	0
1	1	...	1	1

Out is HIGH only if ALL of the N In are HIGH (Principal Mosfet Off). This is a AND gate with N inputs



- Open Drain/Collector Gates -

- the cell used in the Wired AND is an «open drain» gate



- The «open drain gates are obtained by removing the «high» MOS from the standard gate. Doing so, the gate can only drive low.
- Open drain gates are labelled with a star or similar symbol

CPLD Architecture

Prof. Stefano Ricci

UNIVERSITÀ DEGLI STUDI FIRENZE
DINFO
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Updated 02/2025

- CPLD -

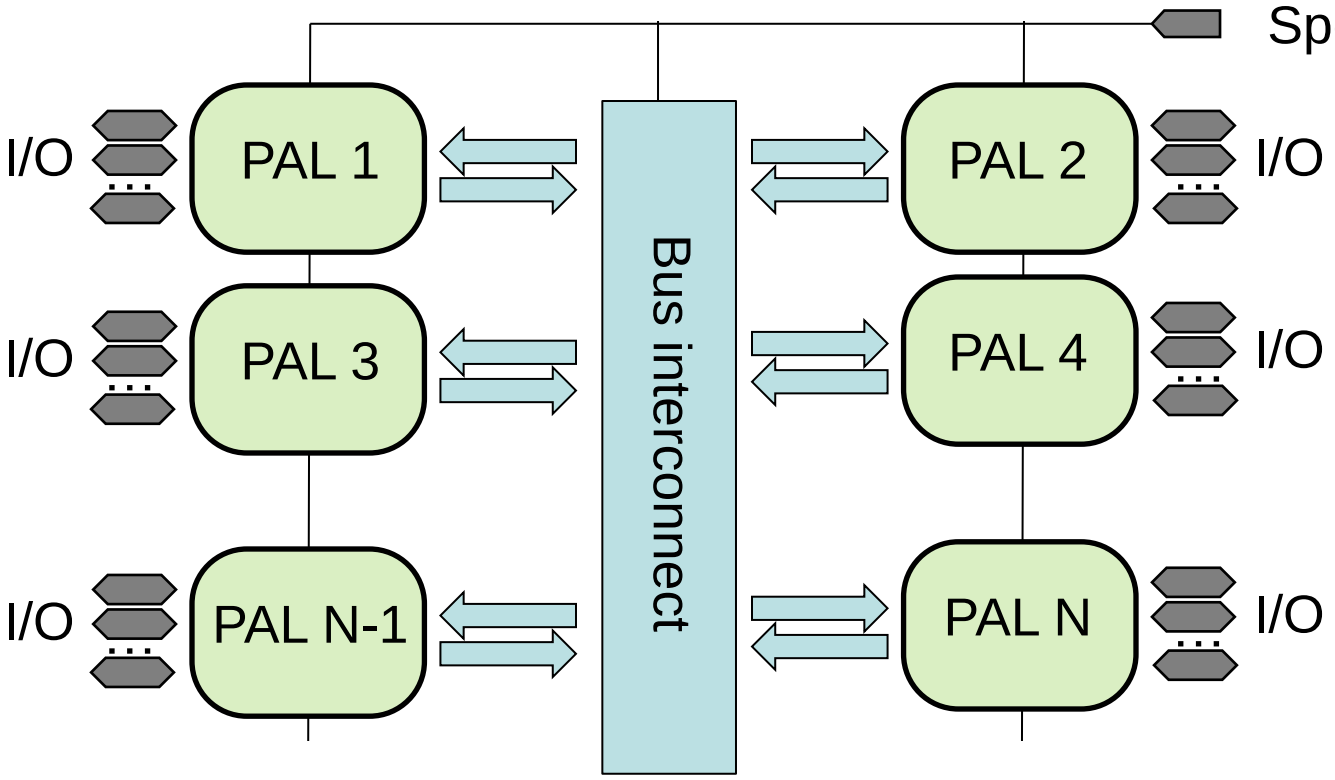
- CPLD: Complex Programmable Logic Device
- CPLDs represent the 'natural' development of the PAL

But with enhanced capacity…

	PAL	CPLD
I/O pin	~ 30	Up to 250
Cells	~ 10	Up to 1700

- CPLD architecture -

- CPLD is basically an array of interconnected PALs



alla programmabilità di ciascuna PAL viene aggiunta tutta la parte di interconnessione => posso decidere come interconnettere ingressi uscite di diverse PAL => ho bus di interconnessione configurabili

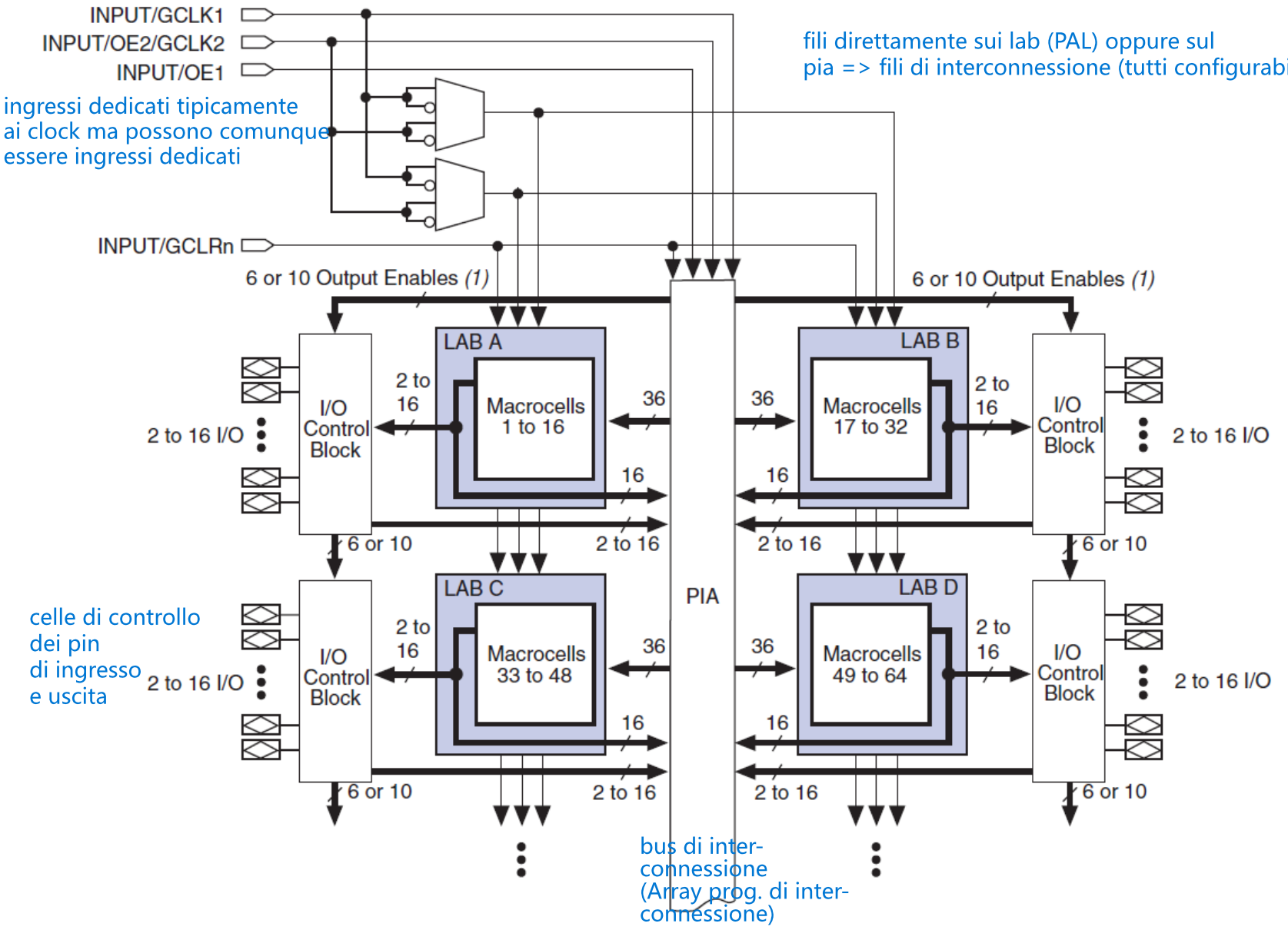
- The interconnections among PALs are programmable, i.e. the user can select which lines are routed to a specific PAL

- CPLD architecture -

la configurabilità di questi dispositivi è legata al fgmos (memoria flash) => una volta configurata una cella questa rimane anche per diverso tempo => una volta che lo accendo questo se configurato risponde subito
viceversa le fpga si basano sulle RAM => all'accensione questa è azzerata e dunque necessità di una fase di caricamento dei dati => "ha bisogno di un po' di tempo per accendersi"
=> per cui l'utilizzo dei due dispositivi cambia a seconda dei requisiti

- Esempio commerciale: Famiglia MAX3000 di Altera
- Tecnologia 0.3 um
- Basata su memoria Flash

- MAX 3000 CPLD - Architettura



fili direttamente sui lab (PAL) oppure sul pia => fili di interconnessione (tutti configurabili)

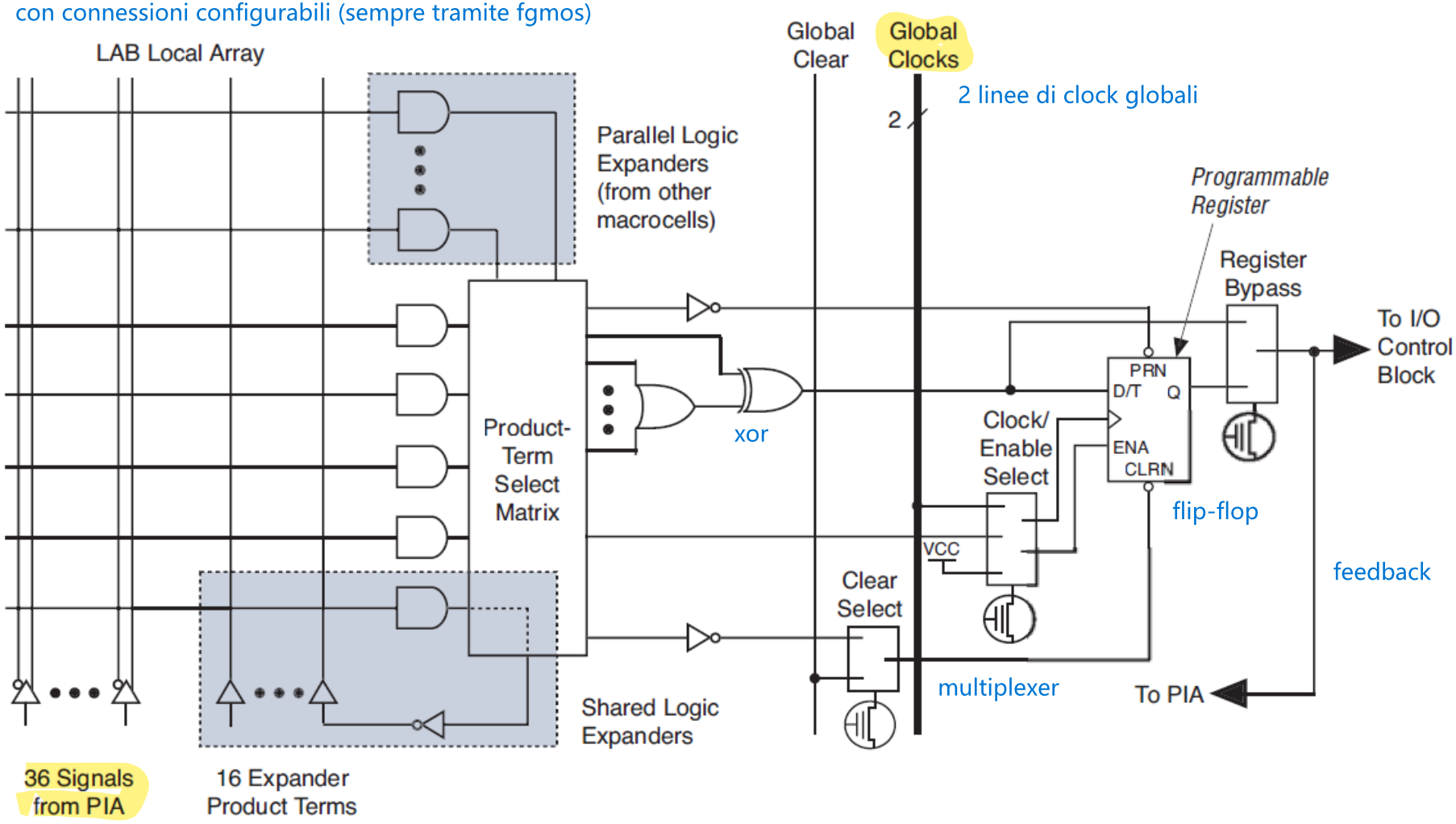
ingressi dedicati tipicamente ai clock ma possono comunque essere ingressi dedicati

celle di controllo dei pin di ingresso e uscita

bus di inter-connessione (Array prog. di inter-connessione)

- MAX 3000 CPLD - Macrocella

matrice di and-or (ingressi dal basso)
con connessioni configurabili (sempre tramite fgmos)

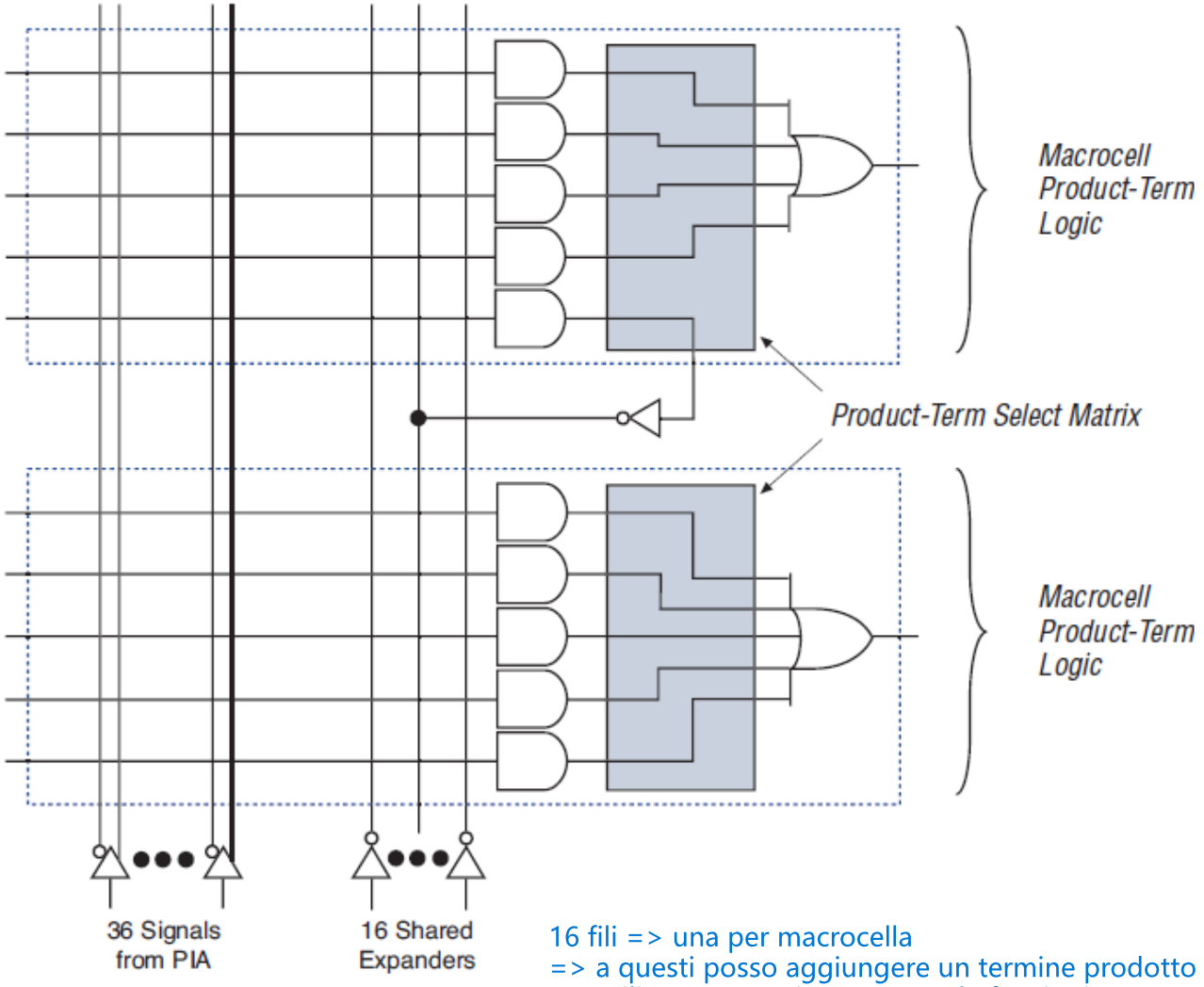


36 fili dal PIA comuni a tutte le macrocelle

- MAX 3000 CPLD – Shareable Expanders

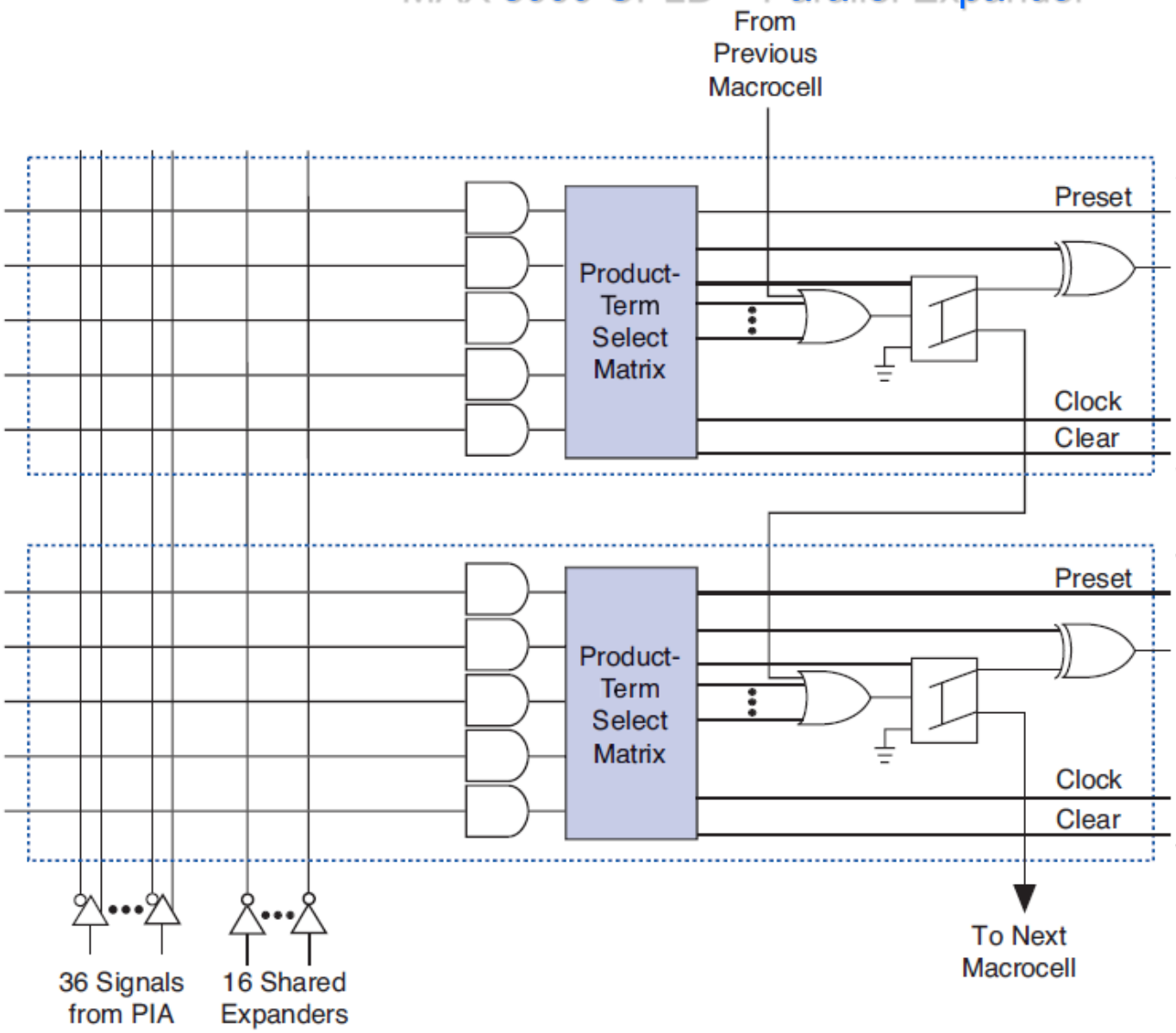
foto riprese dal datasheet del dispositivo

Shareable expanders can be shared by any or all macrocells in an LAB.



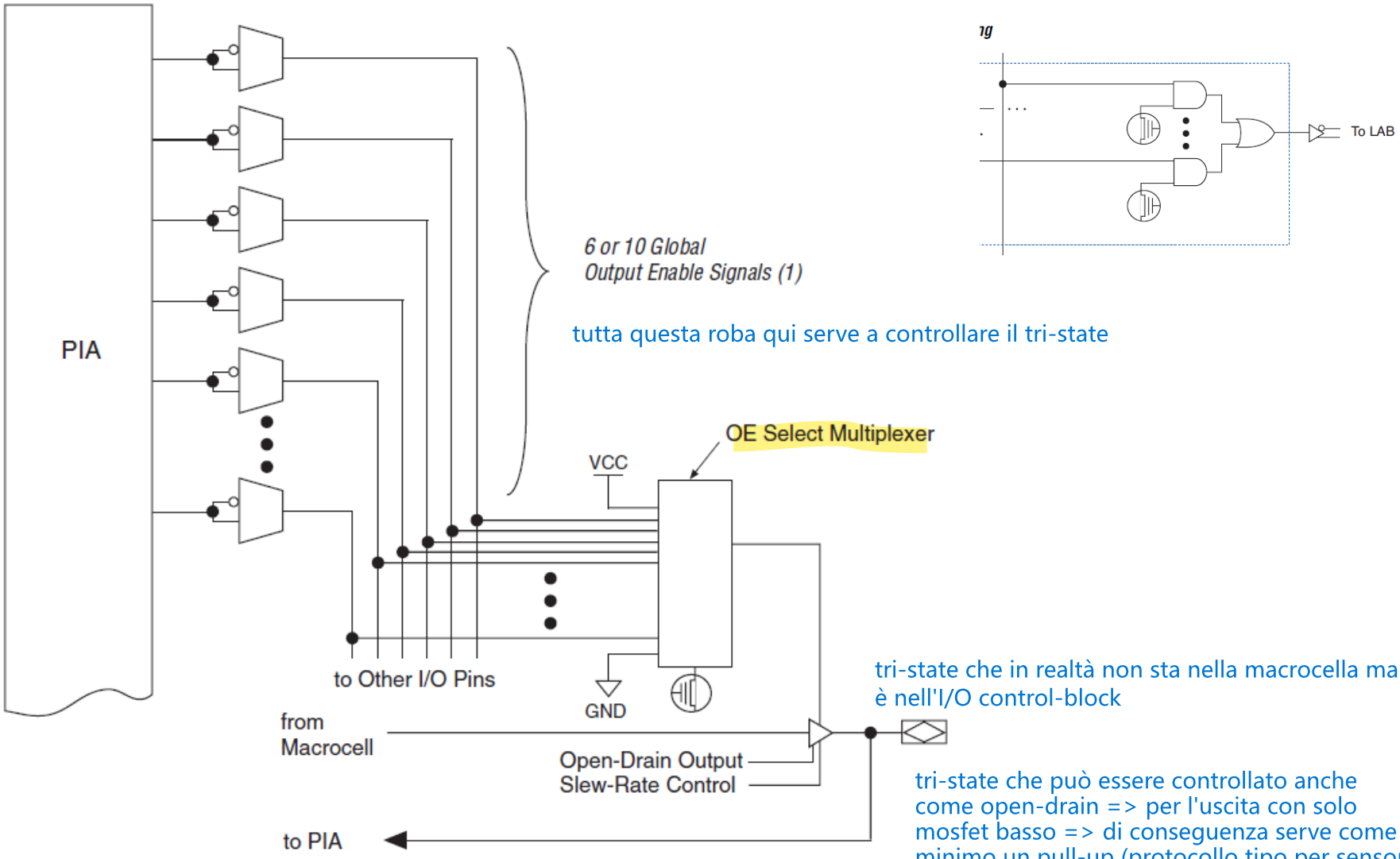
16 fili => una per macrocella
=> a questi posso aggiungere un termine prodotto negato
=> utilizzato come ingresso per le funzioni somma-prodotto

- MAX 3000 CPLD – Parallel Expander



i 5 termini prodotto della cella sopra vengono riproposti sotto (tutti come termini in OR) ciascuna macrocella può prestare i suoi termini alla macrocella successiva

- MAX 3000 CPLD – I/O Blocks



tutta questa roba qui serve a controllare il tri-state

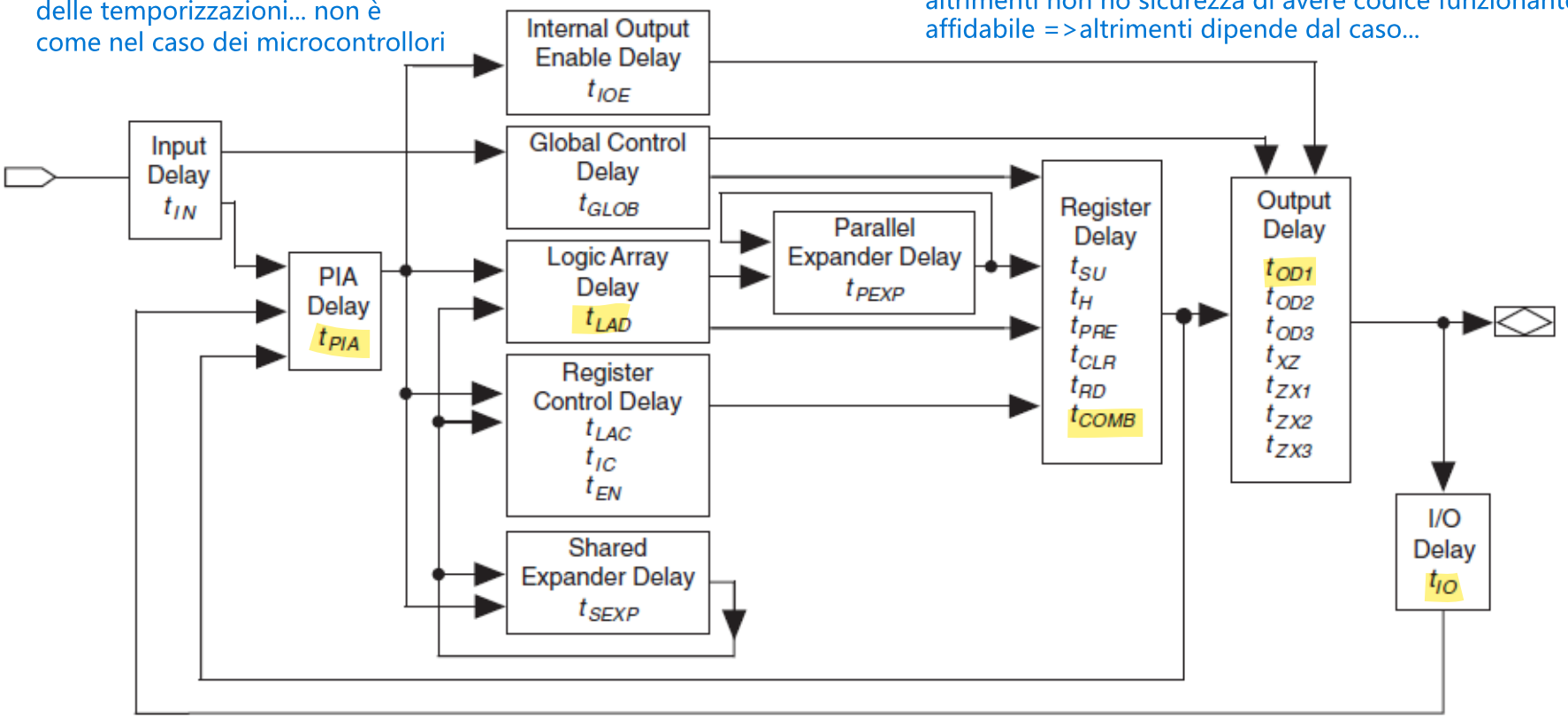
tri-state che in realtà non sta nella macrocella ma è nell'I/O control-block

tri-state che può essere controllato anche come open-drain => per l'uscita con solo mosfet basso => di conseguenza serve come minimo un pull-up (protocollo tipo per sensori oppure I2C)

- Timing MAX 3000 CPLD – Timing model

nei cpld c'è da preoccuparsi delle temporizzazioni... non è come nel caso dei microcontrollori

fare codice per cpld deve tener conto di queste => altrimenti non ho sicurezza di avere codice funzionante e affidabile =>altrimenti dipende dal caso...



software fa in modo automatico i calcoli per il modello di temporizzazione => usando tutte le info riguardanti i ritardi dei vari componenti

- For details and examples see «TimingMax7000.pdf» in moodle

evidenziato il percorso che facciamo nel caso in cui si percorre tutti i passi per cui si vuole riportare in uscita l'ingresso inserito (caso in cui non aggiungo nessuna funzione => ho solo funzione identità)

Table 17. EPM3032A Internal Timing Parameters (Part 1 of 2) Note (1)

Symbol	Parameter	Conditions	Speed Grade						Unit
			–4		–7		–10		
			Min	Max	Min	Max	Min	Max	
t_{IN}	Input pad and buffer delay			0.7		1.2		1.5	ns
t_{IO}	I/O input pad and buffer delay			0.7		1.2		1.5	ns
t_{SEXP}	Shared expander delay			1.9		3.1		4.0	ns
t_{PEXP}	Parallel expander delay			0.5		0.8		1.0	ns
t_{LAD}	Logic array delay			1.5		2.5		3.3	ns
t_{LAC}	Logic control array delay			0.6		1.0		1.2	ns
t_{IOE}	Internal output enable delay			0.0		0.0		0.0	ns
t_{OD1}	Output buffer and pad delay, slow slew rate = off V_{CCIO} = 3.3 V	C1 = 35 pF		0.8		1.3		1.8	ns
t_{OD2}	Output buffer and pad delay, slow slew rate = off V_{CCIO} = 2.5 V	C1 = 35 pF		1.3		1.8		2.3	ns
t_{OD3}	Output buffer and pad delay, slow slew rate = on V_{CCIO} = 2.5 V or 3.3 V	C1 = 35 pF		5.8		6.3		6.8	ns

la scelta dei "tratti" dipende dal tipo di CPLD con cui si ha a che fare

ovviamente i più veloci vanno a prezzi + alti (sono tutte garantite funzionanti)

ocho cambia alimentazione!

abbassando aliment. => + lento

qui invece aggiungo un controllo => + lento... => rallento la velocità d'uscita in modo tale da avere meno rumore

t_{ZX1}	Output buffer enable delay, slow slew rate = off V_{CCIO} = 3.3 V	C1 = 35 pF		4.0		4.0		5.0	ns
t_{ZX2}	Output buffer enable delay, slow slew rate = off V_{CCIO} = 2.5 V	C1 = 35 pF		4.5		4.5		5.5	ns
t_{ZX3}	Output buffer enable delay, slow slew rate = on V_{CCIO} = 2.5 V or 3.3 V	C1 = 35 pF		9.0		9.0		10.0	ns
t_{XZ}	Output buffer disable delay	C1 = 5 pF		4.0		4.0		5.0	ns
t_{SU}	Register setup time tempo di setup per il reg.		1.3		2.0		2.8		ns
t_H	Register hold time		0.6		1.0		1.3		ns
t_{RD}	Register delay tcq			0.7		1.2		1.5	ns
t_{COMB}	Combinatorial delay			0.6		1.0		1.3	ns
t_{IC}	Array clock delay			1.2		2.0		2.5	ns
t_{EN}	Register enable time			0.6		1.0		1.2	ns
t_{GLOB}	Global control delay			0.8		1.3		1.9	ns
t_{PRE}	Register preset time			1.2		1.9		2.6	ns
t_{CLR}	Register clear time			1.2		1.9		2.6	ns
t_{PIA}	PIA delay	*(2)*		0.9		1.5		2.1	ns
t_{LPA}	Low–power adder	*(5)*		2.5		4.0		5.0	ns

- MAX 3000 CPLD -

Table 1. MAX 3000A Device Features

Feature	EPM3032A	EPM3064A	EPM3128A	EPM3256A	EPM3512A
Usable gates	600	1,250	2,500	5,000	10,000
Macrocells	32	64	128	256	512
Logic array blocks	2	4	8	16	32
Maximum user I/O pins	34	66	98	161	208
t_{PD} (ns)	4.5	4.5	5.0	7.5	7.5
t_{SU} (ns)	2.9	2.8	3.3	5.2	5.6
t_{CO1} (ns)	3.0	3.1	3.4	4.8	4.7
f_{CNT} (MHz)	227.3	222.2	192.3	126.6	116.3

riassumendo => avendo in input tutti i tempi mi posso calcolare (in modo automatico) tutti i tempi per i possibili percorsi => e avere così un diagramma di temporizzazioni (notare temporizzazioni indipendenti dal percorso => ovvero ad esempio il tempo di PIA è esattamente lo stesso qualsiasi sia il punto di ingresso oppure di uscita) => questo solo nel caso però di CPLD con abbastanza pochi elementi... altrimenti perdo indipendenza del percorso nel calcolo delle temporizzazioni (caso delle FPGA)

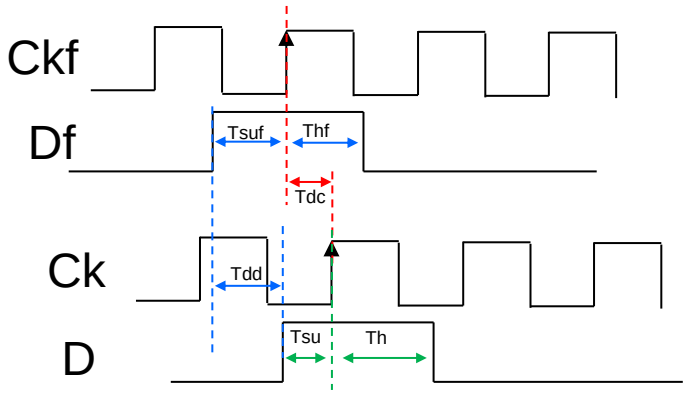
- Examples -

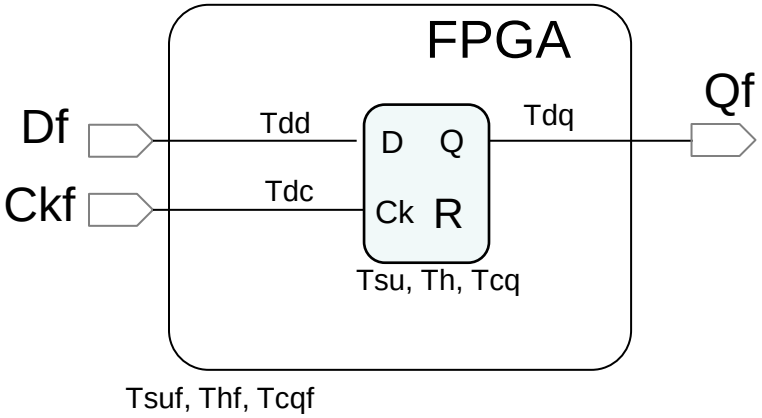
per capire la frequenza massima del circuito guardo il percorso che facciamo seguendo: un registro (tcq) - il tempo di delay per il calcolo della funzione (tpia+tlad) + tempo per setup del registro (tsu) => una volta ottenuto dunque si prende il reciproco

Evaluation of set-up, hold and Ck-to-Q time Tsuf, Thf, Tcqf at FPGA level when Tdd, Tdc, Tdq are the internal data and clock delays

ad esempio per tempo (minimo) di 9,7 ns si ottiene una freq massima di circa 100 mega hz

Tsuf = Tsu+Tdd-Tdc
Thf = Th –Tdd+Tdc
Tcqf = Tdc +Tcq+Tdq





- For more examples see «TimingMax7000.pdf» in moodle

- MAX 3000 CPLD -

Table 3. MAX 3000A Maximum User I/O Pins

Device	44–Pin PLCC	44–Pin TQFP	100–Pin TQFP	144–Pin TQFP	208–Pin PQFP	256-Pin FineLine BGA
EPM3032A	34	34				
EPM3064A	34	34	66			
EPM3128A			80	96		98
EPM3256A				116	158	161
EPM3512A					172	208